ĐẠI HỌC PHENIKAA

TRƯỜNG CÔNG NGHỆ THÔNG TIN PHENIKAA

*****

BÁO CÁO ĐỒ ÁN CƠ SỞ

Nhóm 4

Tìm hiểu về phương pháp hình thức trong công nghệ phần mềm và thực hiện với Hệ thống đăng ký học phần trên Alloy

Giảng viên hướng dẫn: ThS.Vũ Quang Dũng

Sinh viên thực hiện:

Họ và tên	Mã sinh viên	Điểm số
Phạm Thị Phương Anh	23010706	
Trần Thị Ngoãn	23010154	

<Hà Nội, tháng 3 năm 2026>

MỤC LỤC **3**
Chương 1: Tổng quan đề tài **6**
1.1. Lý do chọn đề tài 6
1.2. Mục tiêu và phạm vi nghiên cứu 6
Chương 2: Nền tảng lý thuyết và các nghiên cứu liên quan **8**
2.1. Phương pháp hình thức - Formal Methods 8
2.1.1. Lý thuyết tập hợp và quan hệ (Set Theory & Relations) 8
2.1.2 Logic vị từ bậc nhất 8
2.1.3. Các thuộc tính kiểm chứng(Safety and Liveness properties) 9
2.2 Ngôn ngữ Alloy - Tổng quan kỹ thuật 11
2.2.1 Giới thiệu chung về ngôn ngữ Alloy 11
2.2.2 Các cấu trúc cơ bản của Alloy 11
2.2.3 Ví dụ minh họa cấu trúc Alloy 12
2.3 Các nghiên cứu liên quan (Related Work) 14
2.3.1 Nhóm nghiên cứu sử dụng Alloy 14
2.3.2 Nhóm nghiên cứu về phương pháp hình thức trong kỹ nghệ phần mềm 15
2.3.3 Nhóm nghiên cứu sử dụng các ngôn ngữ hình thức khác 16
Chương 3: Đặc tả hình thức và mô hình Alloy **18**
3.1. Phương pháp đặc tả 18
3.1.2 Mức độ trừu tượng của mô hình 18
3.1.3 Chiến lược kiểm chứng 19
3.2. Mô hình hóa thực thể và quan hệ (Signatures và Relations) 20
3.2.1 Mô hình thực thể và quan hệ cơ sở 20
- Ví dụ instance 21
- Multiplicity: Thuộc tính capacity 22
Thuộc tính prereq 22
- Ý nghĩa thực tế: registers mô tả quan hệ đăng ký lớp học giữa Student và Class. Quan hệ này cho biết:Sinh viên nào đăng ký lớp học nào trong hệ thống. 23
- Code Alloy 23
3.2.2 Mô hình thực thể và quan hệ mở rộng 24
a. Mở rộng Signature CancelReport (Báo cáo hủy học phần) 24
3.3 Đặc tả tiên đề và bất biến (Facts & Invariants) 28
3.3.1 Ràng buộc của mô hình cơ sở (NosharedBooks) 28
3.3.2 Ràng buộc của mô hình mở rộng 28
3.4 Đặc tả vị từ và hoạt động (Predicates & Operations) 31
a. Predicate: registerCourse **31**
+ Điều kiện 1: Sinh viên chưa đăng ký học phần 33
+ Điều kiện 2: Sinh viên chưa vượt quá số học phần cho phép 33
3.5 Đặc tả assertion và kiểm chứng (Assertions & Verification) 34
3.5.1 Đặc tả assertion và kiểm chứng cơ sở (RegistrationSafety) 34

3.5.2 Các đặc tả assertion và kiểm chứng mở rộng 35
Chương 4: Phân tích thuật toán & ràng buộc 39
4.1 Đặc tả thuật toán 39
4.1.1. Mô tả bài toán 40
4.1.2. Pseudocode 40
4.1.3.Chứng minh tính đúng đắn (Correctness Proof) 41
4.1.4. Phân tích độ phức tạp 44
4.1.5. Mô hình hóa thuật toán trong Alloy 45
4.2. Đặc tả ràng buộc 46
4.2.1 Định nghĩa bài toán ràng buộc (CSP) 46
4.2.2 Phân loại ràng buộc 47
4.2.3 Lan truyền ràng buộc và tính nhất quán (Propagation & Consistency) 47
Chương 5: TÌNH HUỐNG (SCENARIOS) VÀ KIỂM CHỨNG 49
5.1. Thiết kế tình huống kiểm chứng 49
5.2. Phân tích Counterexample 52
5.3. Phân tích Scope và giới hạn kiểm chứng 54
Chương 6: Kết quả, thảo luận và hạn chế 57
6.1 Tổng hợp kết quả kiểm 57
6.2 Thảo luận sau kết quả 57
Chương 7. Kết luận và hướng phát triển 58
7.1 Kết luận nghiên cứu 58
7.2 Đóng góp của đề tài 59
7.2.1 Đóng góp đề tài 59
7.2.2 Hạn chế của nghiên cứu 59
7.3 Hướng phát triển trong tương lai 59
Tài Liệu tham khảo 60

BẢNG PHÂN CÔNG CÔNG VIỆC

Họ và tên	Mã sinh viên	Đóng góp	Hoàn Thành
Phạm Thị Phương Anh	23010706	Xây dựng code Alloy Viết và hoàn thiện báo cáo chương 1, 4, 5, 7	100%
Trần thị Ngoãn	23010154	Xây dựng code Alloy Viết và hoàn thiện báo cáo chương 2, 3, 6, 7	100%

Chương 1: Tổng quan đề tài

1.1. Lý do chọn đề tài

Trong bối cảnh công nghệ thông tin phát triển mạnh mẽ, các hệ thống phần mềm ngày càng phức tạp và đóng vai trò quan trọng trong nhiều lĩnh vực, đặc biệt là trong quản lý giáo dục. Tuy nhiên, nhiều lỗi nghiêm trọng thường xuất phát từ sai sót trong thiết kế và logic hệ thống, chỉ được phát hiện khi phần mềm đã đưa vào vận hành, gây tốn kém chi phí sửa chữa và ảnh hưởng đến độ tin cậy.

Phương pháp hình thức (Formal Methods) ra đời như một hướng tiếp cận khoa học nhằm nâng cao chất lượng phần mềm thông qua việc đặc tả và kiểm chứng hệ thống bằng các công cụ toán học ngay từ giai đoạn thiết kế. Nhờ đó, các lỗi tiềm ẩn có thể được phát hiện sớm, góp phần giảm thiểu rủi ro trong quá trình phát triển và triển khai hệ thống.

Hệ thống đăng ký học phần là một nghiệp vụ quen thuộc trong các trường đại học, chứa nhiều ràng buộc logic phức tạp như giới hạn số tín chỉ, điều kiện tiên quyết môn học, xung đột thời khóa biểu, số lượng sinh viên tối đa mỗi lớp..v.v. Đây là đối tượng phù hợp để áp dụng ngôn ngữ đặc tả hình thức Alloy nhằm mô hình hóa và kiểm chứng các ràng buộc đó một cách chặt chẽ và có hệ thống.

Vì vậy, nhóm em quyết định chọn đề tài này nhằm tìm hiểu lý thuyết về phương pháp hình thức, đồng thời thực hành áp dụng công cụ Alloy Analyzer vào một bài toán thực tế, qua đó nâng cao kỹ năng tư duy hệ thống, thiết kế đặc tả phần mềm và kiểm chứng tính đúng đắn của mô hình

1.2. Mục tiêu và phạm vi nghiên cứu

- **Mục tiêu:** Đề tài được thực hiện với mục tiêu tìm hiểu và làm rõ vai trò của phương pháp hình thức trong công nghệ phần mềm, đồng thời áp dụng phương pháp này nhằm mô hình hóa và kiểm chứng hệ thống đăng ký học phần thông qua công cụ Alloy. Đề tài được nghiên cứu với mong muốn giúp người đọc hiểu hơn về giá trị của phương pháp hình thức trong việc phát hiện và ngăn ngừa các lỗi logic từ những giai đoạn đầu khi bắt tay vào việc xây dựng một hệ thống phần mềm. Để đạt được chúng, đồ án cần tập trung vào từng mục tiêu cụ thể :
    - Tìm hiểu và nghiên cứu các khái niệm cơ bản của phương pháp hình thức với hai nội dung chính bao gồm: đặc tả hình thức (Formal Specification) và xác minh hình thức(Formal Verification), cũng như vị trí của phương pháp này trong quy trình phát triển phần mềm (SDLC).
    - Tìm hiểu, làm quen với Alloy và công cụ Alloy Analyzer, nắm vững cú pháp, cách xây dựng mô hình và kiểm chứng các ràng buộc.
    - Áp dụng Alloy để xây dựng mô hình hình thức cho hệ thống đăng ký học phần, đặc tả các thực thể, quan hệ và ràng buộc nghiệp vụ của hệ thống.

- Thực hiện kiểm chứng các thuộc tính an toàn của hệ thống, ví dụ như đảm bảo sinh viên không thể đăng ký hai học phần trùng lịch, hay không vượt quá số tín chỉ tối đa cho phép trong một học kỳ.
- Tiến hành phân tích các phản ví dụ (counterexample), cho Alloy Analyzer hiển thị sơ đồ nhằm phát hiện các lỗi logic tiềm ẩn trong thiết kế hệ thống.
- Đánh giá hiệu quả của phương pháp hình thức bằng cách so sánh với các phương pháp kiểm thử phần mềm truyền thống, nhận xét về ưu, nhược điểm và khả năng ứng dụng thực tế.

- **Phạm vi nghiên cứu:**
  - Đề tài tập chung vào nghiên cứu phương pháp hình thức trong công nghệ phần mềm ở mức độ cơ bản và áp dụng vào bài toán Hệ thống đăng ký học phần trong phạm vi một trường đại học. Hệ thống được xem xét dưới góc độ mô hình logic, không phải một phần mềm triển khai hoàn chỉnh.
  - Về đối tượng sử dụng, hệ thống gồm hai nhóm chính: sinh viên và cán bộ quản lý đào tạo. Sinh viên sử dụng hệ thống để đăng ký, hủy đăng ký học phần và kiểm tra các điều kiện ràng buộc như tiên quyết, giới hạn số tín chỉ hoặc trùng lịch học. Cán bộ quản lý có vai trò thiết lập danh sách học phần, quy định sĩ số tối đa và điều kiện đăng ký.
  - Về môi trường sử dụng, hệ thống được giả định hoạt động trong môi trường trực tuyến của nhà trường và được sử dụng chủ yếu trong các đợt đăng ký học phần theo từng học kỳ.
  - Trong phạm vi thực nghiệm, đề tài chỉ mô hình hóa một phần hệ thống liên quan trực tiếp đến quá trình đăng ký học phần và các ràng buộc logic kèm theo. Các chức năng khác như quản lý học phí, quản lý điểm, bảo mật hệ thống hay triển khai cơ sở dữ liệu thực tế sẽ không được đề cập trong đề tài. Cần xác định rõ nội dung chính của đề tài là xây dựng mô hình logic và kiểm chứng các thuộc tính an toàn bằng Alloy, tránh nhầm lẫn với việc triển khai một phần mềm hoàn chỉnh hay viết trương trình trên các ngôn ngữ lập trình khác như C, Java hay Python.

Chương 2: Nền tảng lý thuyết và các nghiên cứu liên quan

2.1. Phương pháp hình thức - Formal Methods

Phương pháp hình thức(Formal Methods) là tập hợp các kỹ thuật toán học bao gồm logic, lý thuyết tập hợp và đồ thị sử dụng cho việc đặc tả, phát triển và kiểm định các hệ thống phần mềm và phần cứng. Mục tiêu cốt lõi của phương pháp hình thức là giải quyết bài toán về tính đúng đắn của phần mềm ngay từ giai đoạn đặc tả và thiết kế dựa trên cơ sở lý thuyết chặt chẽ nhằm mục đích giảm bớt sự phụ thuộc vào trực giác và phán đoán của con người trong việc đánh giá lập luận thiết kế.

Khác với các phương pháp kiểm thử truyền thống, thường dựa trên việc chạy chương trình với một số tập dữ liệu và các kịch bản cụ thể, phương pháp hình thức lại tiếp cận bài toán ở dạng mô hình. Nhờ đó, các lỗi logic tiềm ẩn có thể được phát hiện sớm trước khi hệ thống được triển khai thực tế. Trong đề tài này, phương pháp hình thức được sử dụng để mô hình hóa và kiểm chứng chức năng mượn sách của hệ thống quản lý thư viện thông qua công cụ Alloy.

2.1.1. Lý thuyết tập hợp và quan hệ (Set Theory & Relations)

Lý thuyết tập hợp và quan hệ là nền tảng quan trọng trong việc mô hình hóa hệ thống bằng phương pháp hình thức. Trong cách tiếp cận này, không tồn tại khái niệm đối tượng đơn lẻ mà mọi thực thể đều được coi là các tập hợp con của tích Đề - các (Cartesian product) giữa các tập hợp.

Trong hệ thống đăng ký học phần của đề tài, ta có thể mô tả các thành phần chính như sau:

- B = tập hợp tất cả các học phần (Course).
- M = tập hợp tất cả các sinh viên (Student).
- borrowed_by $\subseteq$ M x B: Biểu diễn trạng thái đăng ký học phần của các sinh viên.

Quan hệ borrowed_by là tập hợp các cặp <m, b> biểu diễn việc sinh viên, đang đăng ký học phần b. Việc sử dụng mô hình tập hợp này giúp hệ thống được mô tả một cách rõ ràng và dễ dàng chuyển đổi sang mô hình Alloy trong các phần tiếp theo.

2.1.2 Logic vị từ bậc nhất

Logic vị từ bậc nhất (First-Order Predicate Logic) là nền tảng lý thuyết quan trọng của nhiều phương pháp hình thức. Logic vị từ cho phép mô tả các thuộc tính của hệ thống thông qua các biến, quan hệ và lượng từ phổ biến bao gồm:

- $\forall$ (forall): với mọi phần tử
- $\exists$ (exists): tồn tại ít nhất một phần tử

Các lượng từ này cho phép mô tả các ràng buộc logic trong hệ thống một cách chính xác.

Ví dụ minh họa: Để diễn đạt quy tắc " Mỗi học phần sinh viên chỉ được đăng ký tối đa một lần", ta sử dụng lượng từ ($\forall$) kết hợp với logic định lượng:

$$\forall b \in Course, |\{m \in Student \mid (m,b) \in borrowed_by\}| \leq 1$$

Để thuận tiện cho việc mô tả các ràng buộc logic của hệ thống, một số ký hiệu toán học được sử dụng xuyên suốt quá trình mô hình hóa được tổng hợp trong bảng 2.1. Các ký hiệu này chủ yếu xuất phát từ logic vị từ và lý thuyết tập hợp, là nền tảng cho việc mô hình hóa bài toán sau này.

Ký hiệu	Ý nghĩa	Ví dụ ứng dụng trong đề tài
$\forall$	với mọi(for all)	$\forall$S1,S2 $\in$ Student: áp dụng cho mọi sinh viên
$\exists$x	tồn tại(có ít nhất một)	$\exists$m $\in$ Student: Tồn tại ít nhất một sinh viên
$\exists$!x	Tồn tại duy nhất	$\exists$!m $\in$ Student: chỉ có đúng một sinh viên
$\nexists$x	Không tồn tại	$\nexists$m $\in$ Student: Không tồn tại sinh viên nào
$\in$	Thuộc về một tập hợp	b $\in$ m.đăng ký: học phần b thuộc tập hợp học phần mà m đang đăng
$\Rightarrow$	kéo theo (IMPLIES)	(b $\in$ m_1.đăng ký $\wedge$ b $\in$ m_2.đăng ký) $\Rightarrow$(m_1=m_2)
$\subseteq$	tập con	borrowed_by $\subseteq$ Student $\times$ Course
$\times$	tích Descartes	Student $\times$ Course : không gian các cặp đăng ký
| A|	số phân tử của tập	|m.đăng ký| : số lượng học phần sinh viên đã đăng ký
$\leq$	Quan hệ nhỏ hơn hoặc bằng	|m.đăng ký|$\leq$16: mỗi sinh viên đăng ký tối đa 16 tín chỉ

Bảng 2.1: Các ký hiệu toán học sử dụng trong mô hình

2.1.3. Các thuộc tính kiểm chứng(Safety and Liveness properties)

Trong phương pháp hình thức, việc xác định các thuộc tính là bước quan trọng nhằm mô tả các tính chất mà hệ thống bắt buộc phải thỏa mãn, từ đó làm cơ sở cho

việc chứng minh tính đúng đắn của thiết kế.. Các thuộc tính được chia làm 2 loại chính là thuộc tính an toàn và thuộc tính sống.

a. Thuộc tính an toàn (Safety Properties)

Thuộc tính an toàn là các thuộc tính đảm bảo rằng những trạng thái không mong muốn sẽ không bao giờ xảy ra trong suốt quá trình hoạt động của hệ thống. Nói cách khác, nếu một thuộc tính an toàn bị vi phạm thì hệ thống đã rơi vào trạng thái lỗi tại một thời điểm cụ thể.

Đặc điểm của thuộc tính an toàn:

- Có thể bị vi phạm tại một thời điểm xác định.

- Khi đã vi phạm thì không thể khắc phục trong quá trình thực thi.

- Thường được dùng để kiểm soát lỗi, xung đột và các ràng buộc nghiệp vụ.

Ví dụ về thuộc tính an toàn trong hệ thống đăng ký học phần: Mỗi sinh viên không được đăng ký hai lớp học phần trùng thời gian học hoặc số lượng sinh viên đăng ký không vượt quá sĩ số tối đa của lớp học phần. Sinh viên không được đăng ký học phần khi chưa hoàn thành học phần tiên quyết và không được đăng ký trùng cùng một học phần trong cùng một học kỳ.

Trong Alloy, các thuộc tính an toàn thường được biểu diễn bằng:

- Các fact (ràng buộc luôn đúng)

- Hoặc assert để kiểm tra xem có tồn tại phản ví dụ vi phạm thuộc tính hay không.

Thuộc tính an toàn rất phù hợp với phương pháp Model Checking, vì công cụ có thể tìm ra phản ví dụ cụ thể nếu thuộc tính bị vi phạm.

b. Thuộc tính sống (Liveness Properties)

Thuộc tính sống là các thuộc tính đảm bảo rằng một điều mong muốn nào đó cuối cùng sẽ xảy ra trong quá trình hoạt động của hệ thống. Khác với thuộc tính an toàn, thuộc tính sống không yêu cầu điều kiện phải đúng tại mọi thời điểm, mà yêu cầu hệ thống phải có khả năng tiến triển theo thời gian.

Đặc điểm của thuộc tính sống:

- Không bị vi phạm ngay tại một thời điểm cụ thể.

- Liên quan đến sự tiến triển hoặc hoàn tất của hệ thống.

- Thường gắn với hành vi động và chuỗi sự kiện theo thời gian.

Ví dụ về thuộc tính sống trong hệ thống đăng ký học phần: Mọi yêu cầu đăng ký hợp lệ của sinh viên cuối cùng sẽ được xử lý. Sinh viên đủ điều kiện đăng ký học phần thì sẽ đăng ký thành công. Hệ thống luôn cho phép sinh viên tiếp tục đăng ký cho đến khi kết thúc thời gian đăng ký.

Trong thực tế, việc kiểm chứng thuộc tính sống thường phức tạp hơn thuộc tính an toàn, vì nó liên quan đến yếu tố thời gian và trạng thái tương lai. Do đó, các thuộc tính sống thường được kiểm chứng hiệu quả hơn bằng:

- Model Checking nâng cao

- Hoặc Theorem Proving trong các hệ thống phức tạp

Trong Alloy, do tập trung vào mô hình tĩnh và phạm vi hữu hạn, các thuộc tính sống thường được biểu diễn dưới dạng các điều kiện tồn tại hoặc các ràng buộc đảm bảo khả năng xảy ra trong một phạm vi kiểm tra nhất định.

2.2 Ngôn ngữ Alloy - Tổng quan kỹ thuật

2.2.1 Giới thiệu chung về ngôn ngữ Alloy

Ngôn ngữ đặc tả Alloy là một trong những công cụ tiêu biểu của phương pháp hình thức nhẹ, được sử dụng rộng rãi trong việc mô hình hóa và phân tích các hệ thống phần mềm có nhiều ràng buộc logic. Alloy cho phép mô tả cấu trúc hệ thống một cách chính xác, loại bỏ sự mơ hồ trong yêu cầu và hỗ trợ kiểm chứng tự động thông qua công cụ Alloy Analyzer. Việc lựa chọn Alloy trong đề tài này nhằm mục đích nghiên cứu cách áp dụng phương pháp hình thức vào hệ thống đăng ký học phần, từ đó đánh giá khả năng phát hiện sớm lỗi logic trong giai đoạn phân tích và thiết kế.

Alloy thường được sử dụng trong các giai đoạn:

- Phân tích yêu cầu hệ thống
- Thiết kế kiến trúc và mô hình dữ liệu
- Kiểm tra tính nhất quán và đầy đủ của thiết kế

Trong đồ án này, Alloy được sử dụng để mô hình hóa các thành phần và quy tắc nghiệp vụ của hệ thống đăng ký học phần chẳng hạn như:

- Sinh viên đăng ký môn học
- Giới hạn số lượng sinh viên trong một lớp
- Tránh trùng lịch học
- Điều kiện tiên quyết của học phần.

2.2.2 Các cấu trúc cơ bản của Alloy

Ngôn ngữ Alloy được xây dựng dựa trên nền tảng Logic bậc nhất (First-order logic) và Logic quan hệ (Relational logic). Về mặt lý thuyết, cấu trúc của một mô hình Alloy tập

trung vào việc định nghĩa các tập hợp và mối quan hệ giữa chúng thông qua các thành phần cốt lõi. Alloy cung cấp một tập các cấu trúc ngôn ngữ giúp mô hình hóa hệ thống. Bảng dưới đây tóm tắt các cấu trúc chính được sử dụng trong đề tài.Các cấu trúc này giúp mô hình hóa hệ thống dưới dạng tập hợp (set) và quan hệ (relation), phù hợp với bản chất của nhiều bài toán trong kỹ thuật phần mềm.

Cấu trúc Alloy	Ý nghĩa	Khi sử dụng trong đề tài
sig A	Khai báo một tập hợp các đối tượng	Student, Course, Instructor
sig B extends A	B là tập con của A (kế thừa)	Phân loại user
sig C in A	C là tập con của A nhưng có thể giao nhau	Vai trò linh hoạt
rel: A -> B	Quan hệ giữa hai tập hợp	Student đăng ký Course
fact	Ràng buộc luôn đúng	Quy tắc hệ thống
pred	Vị từ mô tả trạng thái đến thao tác	Đăng ký học phần
assert	Thuộc tính cần kiểm chứng	Kiểm tra tính nhất quán
check	Kiểm tra assert	Tìm counterexample
run	Yêu cầu Alloy tìm một cấu hình hệ thống hợp lệ	Minh họa mô hình
fun	Hàm trả về giá trị	Truy vấn dữ liệu

Bảng 2.2 Bảng tóm tắt các cấu trúc chính được sử dụng trong đề tài

2.2.3 Ví dụ minh họa cấu trúc Alloy

Để minh họa cách sử dụng Alloy trong việc mô hình hóa hệ thống, ta xét một ví dụ đơn giản liên quan đến chức năng đăng ký học trong hệ thống đăng ký học phần.

```
-- Khai báo các thực thể
sig Student {}      -- Sinh viên
sig Course {        -- Học phần
   capacity: Int,  -- Sức chứa tối đa
   prereq: set Course  -- Môn tiên quyết
}
-- Quan hệ đăng ký học phần
sig Enrollment {
   student: Student,
   course: Course
}
--  Mỗi sinh viên chỉ đăng ký tối đa 1 lần cho mỗi môn
fact UniqueEnrollment {
   all s: Student, c: Course |
      lone e: Enrollment |
         e.student = s and e.course = c
}
-- Số sinh viên đăng ký không vượt quá sức chứa
fact CapacityConstraint {
   all c: Course |
      #(Enrollment.course & c) <= c.capacity
}
--  Sinh viên chỉ được đăng ký nếu đã học môn tiên quyết
sig Passed {
   student: Student,
   course: Course
}
fact PrerequisiteConstraint {
   all e: Enrollment |
      all p: e.course.prereq |
         some pa: Passed |
            pa.student = e.student and pa.course = p
}
-- Thuộc tính cần kiểm chứng
assert NoDuplicateEnrollment {
   all e1, e2: Enrollment |
      (e1.student = e2.student and e1.course = e2.course)
      => e1 = e2
}
-- Kiểm chứng với scope = 5
check NoDuplicateEnrollment for 5
```

Hình 2.4 Ví dụ cấu trúc Alloy - mô hình đăng ký học

Giải thích các thành phần chính:

- **sig Student {}**: Khai báo tập hợp các sinh viên trong hệ thống.
- **sig Course { capacity: Int, prereq: set Course }**: Khai báo tập hợp các học phần. Trong đó:
    + **capacity:** biểu thị sức chứa tối đa của học phần, tức là số sinh viên tối đa được phép đăng ký.

    + **prereq:** biểu thị tập các môn tiên quyết của học phần đó. Điều này có nghĩa là trước khi đăng ký học phần này, sinh viên phải hoàn thành các học phần nằm trong tập prereq.

- **sig Enrollment:** Mô tả một bản ghi đăng ký học phần, liên kết giữa một sinh viên (Student) và một học phần (Course).
- Ràng buộc **fact UniqueEnrollment** đảm bảo rằng mỗi sinh viên chỉ có thể đăng ký tối đa một lần cho cùng một học phần.
- Ràng buộc **fact CapacityConstrain**t đảm bảo rằng tổng số sinh viên đăng ký một học phần không vượt quá sức chứa (capacity) của học phần đó.
- sig Passed mô tả tập các môn học mà sinh viên đã hoàn thành
- Ràng buộc **fact PrerequisiteConstraint** đảm bảo rằng một sinh viên chỉ được đăng ký một học phần khi đã hoàn thành tất cả các môn tiên quyết của học phần đó.
- **assert NoDuplicateEnrollment** được sử dụng để kiểm chứng rằng không tồn tại hai bản ghi đăng ký khác nhau cho cùng một cặp (student, course).
- **check NoDuplicateEnrollment for 5** là lệnh yêu cầu Alloy Analyzer kiểm chứng thuộc tính NoDuplicateEnrollment trong một phạm vi hữu hạn (scope) Quá trình này tự động quét không gian trạng thái. Kết quả trả về thông báo "No counterexample found" (UNSAT), chứng minh rằng mô hình hoàn toàn an toàn và logic phân quyền đã được thiết kế đúng trong giới hạn.

2.3 Các nghiên cứu liên quan (Related Work)

Trong lĩnh vực phương pháp hình thức trong công nghệ phần mềm, nhiều nghiên cứu đã sử dụng các ngôn ngữ đặc tả như Alloy, TLA+, Z notation và các công cụ kiểm chứng tự động nhằm phát hiện lỗi thiết kế ngay từ giai đoạn mô hình hóa. Các nghiên cứu này có thể được phân loại thành ba nhóm chính:

- Nghiên cứu sử dụng Alloy để mô hình hóa và kiểm chứng hệ thống
- Nghiên cứu về ứng dụng phương pháp hình thức trong kỹ nghệ phần mềm
- Nghiên cứu sử dụng các ngôn ngữ hình thức khác như TLA+, Z

2.3.1 Nhóm nghiên cứu sử dụng Alloy

Alloy là một ngôn ngữ đặc tả hình thức dựa trên logic quan hệ và logic vị từ bậc nhất, cho phép mô hình hóa các cấu trúc phần mềm và tự động kiểm chứng bằng Alloy Analyzer.

- Nghiên cứu của Jackson: Công trình của Daniel Jackson giới thiệu Alloy như một ngôn ngữ mô hình hóa quan hệ giúp khám phá thiết kế phần mềm và tự động phát hiện lỗi trong mô hình. Alloy cho phép đặc tả các ràng buộc logic của hệ thống và sử dụng bộ giải SAT để tìm instance hoặc phản ví dụ. Nghiên cứu cho thấy Alloy có thể phát hiện lỗi thiết kế trong các hệ thống quan trọng như giao thức mạng hoặc hệ thống an toàn trước khi triển khai thực tế.

- Nghiên cứu về kiểm chứng kiến trúc hệ thống phân tán: Một nghiên cứu về mô hình kiến trúc hệ thống phân tán đã sử dụng Alloy để đặc tả các mô hình truyền thông như:

+ Message passing
+ Remote procedure call
+ Distributed shared memory

Sau đó Alloy Analyzer được dùng để kiểm chứng tính đúng đắn của các ràng buộc kiến trúc và đảm bảo tính nhất quán giữa các thành phần hệ thống. Kết quả cho thấy phương pháp hình thức giúp phát hiện các vi phạm kiến trúc ngay trong giai đoạn thiết kế

- Nghiên cứu về kiểm chứng dữ liệu hướng đối tượng: Một nghiên cứu gần đây đề xuất khung kiểm chứng dữ liệu sự kiện hướng đối tượng (Object-centric Event Data) bằng Alloy. Mô hình Alloy được sử dụng để đặc tả:

+ Quan hệ giữa các đối tượng
+ Các ràng buộc về thời gian
+ Các ràng buộc toàn vẹn dữ liệu

Kết quả cho thấy việc sử dụng Alloy giúp phát hiện các phụ thuộc dữ liệu và lỗi toàn vẹn trước khi triển khai hệ thống thực tế.

- Nghiên cứu về phân tích mô hình Alloy: Một nghiên cứu khác đã xây dựng dataset các mô hình Alloy và phân tích độ phức tạp của chúng.Nghiên cứu cho thấy việc xây dựng mô hình Alloy thường mang tính lặp và khám phá, trong đó người dùng liên tục chỉnh sửa mô hình để loại bỏ lỗi và cải thiện đặc tả.Điều này chứng minh Alloy là công cụ hữu ích trong giai đoạn thiết kế và kiểm chứng sớm của hệ thống phần mềm.

2.3.2 Nhóm nghiên cứu về phương pháp hình thức trong kỹ nghệ phần mềm

Một số nghiên cứu tập trung vào sử dụng phương pháp hình thức để đặc tả và kiểm chứng yêu cầu hệ thống.

Ví dụ, một nghiên cứu về xác định lỗ hổng kiến trúc phần mềm đã sử dụng đặc tả logic hình thức để mô hình hóa các yêu cầu bảo mật và kiểm chứng chúng trên mô hình hệ thống. Phương pháp này cho phép:

- Mô hình hóa các lỗ hổng kiến trúc
- Kiểm chứng các thuộc tính an toàn

- Phát hiện các lỗi thiết kế trong giai đoạn đầu của phát triển phần mềm.

Kết quả nghiên cứu cho thấy việc áp dụng phương pháp hình thức có thể giảm rủi ro lỗi nghiêm trọng trong các hệ thống phức tạp.

2.3.3 Nhóm nghiên cứu sử dụng các ngôn ngữ hình thức khác

Ngoài Alloy, nhiều ngôn ngữ đặc tả khác cũng được sử dụng trong kiểm chứng hệ thống. TLA+ là ngôn ngữ đặc tả hình thức được phát triển bởi Leslie Lamport, thường dùng để mô hình hóa hệ thống phân tán và hệ thống đồng thời.

Một nghiên cứu so sánh Alloy và TLA+ cho thấy:

- Alloy mạnh trong mô hình hóa cấu trúc và quan hệ
- TLA+ mạnh trong đặc tả thuộc tính thời gian và hành vi động

Do đó mỗi ngôn ngữ phù hợp với các loại bài toán khác nhau.

Nghiên cứu	Tác giả	Phương pháp / Công cụ	Đặc tả ràng buộc an toàn (Safety)	Sinh phản ví dụ (Counter-example)	Khoảng trống nghiên cứu / Hạn chế
Jackson (Alloy)	Daniel Jackson	Alloy	Có – đặc tả các ràng buộc cấu trúc như quan hệ giữa các thực thể, tính duy nhất và toàn vẹn dữ liệu	Alloy Analyzer sinh instance phản ví dụ khi ràng buộc bị vi phạm	Chỉ kiểm chứng trong phạm vi hữu hạn (bounded scope), khó biểu diễn hành vi thời gian
Communication Model Verification	Nhiều tác giả nghiên cứu hệ thống phân tán	Alloy	Có – mô tả tính nhất quán và các ràng buộc giao tiếp giữa các thành phần hệ thống	Công cụ Alloy sinh mô hình vi phạm để phát hiện lỗi thiết kế	Mô hình hóa giao tiếp phức tạp còn hạn chế, khó mở rộng cho hệ thống lớn
OCED Verification	Các nghiên cứu về Object-Centric Event Data	Alloy	Có – đảm bảo tính toàn vẹn dữ liệu và quan hệ giữa các sự kiện và đối tượng	Công cụ tự động sinh trường hợp dữ liệu không hợp lệ	Có – công cụ tự động sinh trường hợp dữ liệu không hợp lệ

Security Vulnerability Modeling	Nhiều nghiên cứu về phân tích bảo mật	Logic hình thức / Formal Logic	Có – mô tả các thuộc tính bảo mật như kiểm soát truy cập, xác thực và tính bí mật	Công cụ kiểm chứng sinh kịch bản vi phạm hoặc tấn công	Mô hình hóa hệ thống thực tế phức tạp, yêu cầu kiến thức logic cao
Alloy vs TLA+ Study	Nhiều nghiên cứu so sánh phương pháp hình thức	Alloy/TLA+	Có – Alloy đặc tả ràng buộc cấu trúc, TLA+ mô tả thuộc tính hành vi và thời gian	Cả Alloy và TLA+ đều sinh phản ví dụ dưới dạng trace thực thi	Alloy hạn chế về temporal logic; TLA+ khó học và mô hình hóa phức tạp
Đồ án		Alloy Analyzer	Có – đặc tả các ràng buộc như sức chứa lớp học, điều kiện tiên quyết của môn học và tránh đăng ký trùng	Alloy Analyzer sinh instance phản ví dụ để phát hiện trường hợp vi phạm như vượt quá sức chứa hoặc thiếu môn tiên quyết	Mô hình chỉ kiểm chứng ở mức thiết kế logic, chưa tích hợp với hệ thống đăng ký thực tế

Bảng 2.3 So sánh các nghiên cứu liên quan với đồ án

Dựa trên các khoảng trống nghiên cứu được xác định từ các bài nghiên cứu trước, đồ án này tập trung vào việc ứng dụng Alloy để mô hình hóa và kiểm chứng logic nghiệp vụ của hệ thống đăng ký học.

Cụ thể, đồ án hướng tới:

- Xây dựng mô hình hình thức của hệ thống đăng ký học bằng Alloy, bao gồm các thực thể chính như *Sinh viên (Student), Học phần (Course), Lớp học (Class)* và *Đăng ký (Registration)* cùng các quan hệ giữa chúng.

- Đặc tả các ràng buộc nghiệp vụ quan trọng của hệ thống, chẳng hạn như: giới hạn sức chứa của lớp học, điều kiện tiên quyết của học phần, và tránh trùng lặp đăng ký.
- Sử dụng Alloy Analyzer để kiểm chứng tự động các ràng buộc, từ đó phát hiện các trường hợp vi phạm thông qua các instance phản ví dụ (counter-example).
- Phân tích các kết quả kiểm chứng nhằm đánh giá tính đúng đắn của mô hình và xác định các lỗi tiềm ẩn trong thiết kế hệ thống.
-Minh họa tính hiệu quả của phương pháp hình thức Alloy trong việc hỗ trợ thiết kế và kiểm chứng hệ thống phần mềm ở giai đoạn sớm.

Chương 3: Đặc tả hình thức và mô hình Alloy

3.1. Phương pháp đặc tả

Trong đề tài này, hệ thống đăng ký học phần được đặc tả bằng phương pháp hình thức sử dụng ngôn ngữ Alloy. Quá trình xây dựng mô hình được thực hiện theo một phương pháp đặc tả có hệ thống nhằm đảm bảo mô hình rõ ràng, nhất quán và có thể kiểm chứng tự động.

Phần này trình bày cách tiếp cận đặc tả, mức độ trừu tượng của mô hình, và chiến lược kiểm chứng được áp dụng trong nghiên cứu.

3.1.1 Cách tiếp cận đặc tả

Đề tài sử dụng phương pháp đặc tả từ trên xuống (Top-down specification approach).

Trong cách tiếp cận này, mô hình hệ thống được xây dựng theo các bước:

- Xác định các thực thể chính của hệ thống
- Xác định các quan hệ giữa các thực thể
- Định nghĩa các ràng buộc và bất biến hệ thống
- Đặc tả các thao tác hoặc trạng thái của hệ thống

Đối với hệ thống đăng ký học phần, các thực thể chính bao gồm:

- Student – sinh viên
- Course – học phần
- Instructor – giảng viên
- Section – lớp học phần
- Registration – bản ghi đăng ký học phần

Sau khi xác định các thực thể này, mô hình Alloy sẽ được xây dựng bằng cách mô tả:

- Các signature biểu diễn thực thể
- Các relation biểu diễn liên kết giữa các thực thể
- Các fact biểu diễn ràng buộc hệ thống

Cách tiếp cận top-down giúp đảm bảo mô hình phản ánh cấu trúc tổng thể của hệ thống trước khi đi vào chi tiết

3.1.2 Mức độ trừu tượng của mô hình

Mô hình Alloy trong đề tài được xây dựng ở mức trừu tượng (abstract model). Mô hình không mô tả chi tiết triển khai phần mềm, mà chỉ tập trung vào:

- Cấu trúc dữ liệu
- Các quan hệ logic giữa các thực thể
- Các ràng buộc nghiệp vụ (business rules)

Ví dụ một số ràng buộc nghiệp vụ trong hệ thống đăng ký học phần:

- Một sinh viên không được đăng ký cùng một học phần nhiều lần
- Một lớp học phần có giới hạn số lượng sinh viên

- Một sinh viên không được đăng ký hai lớp trùng thời gian

Các ràng buộc này được đặc tả bằng logic quan hệ trong Alloy, thay vì bằng code chương trình. Việc sử dụng mô hình trừu tượng giúp:

- Đơn giản hóa hệ thống
- Tập trung vào tính đúng đắn logic
- Giảm độ phức tạp khi kiểm chứng

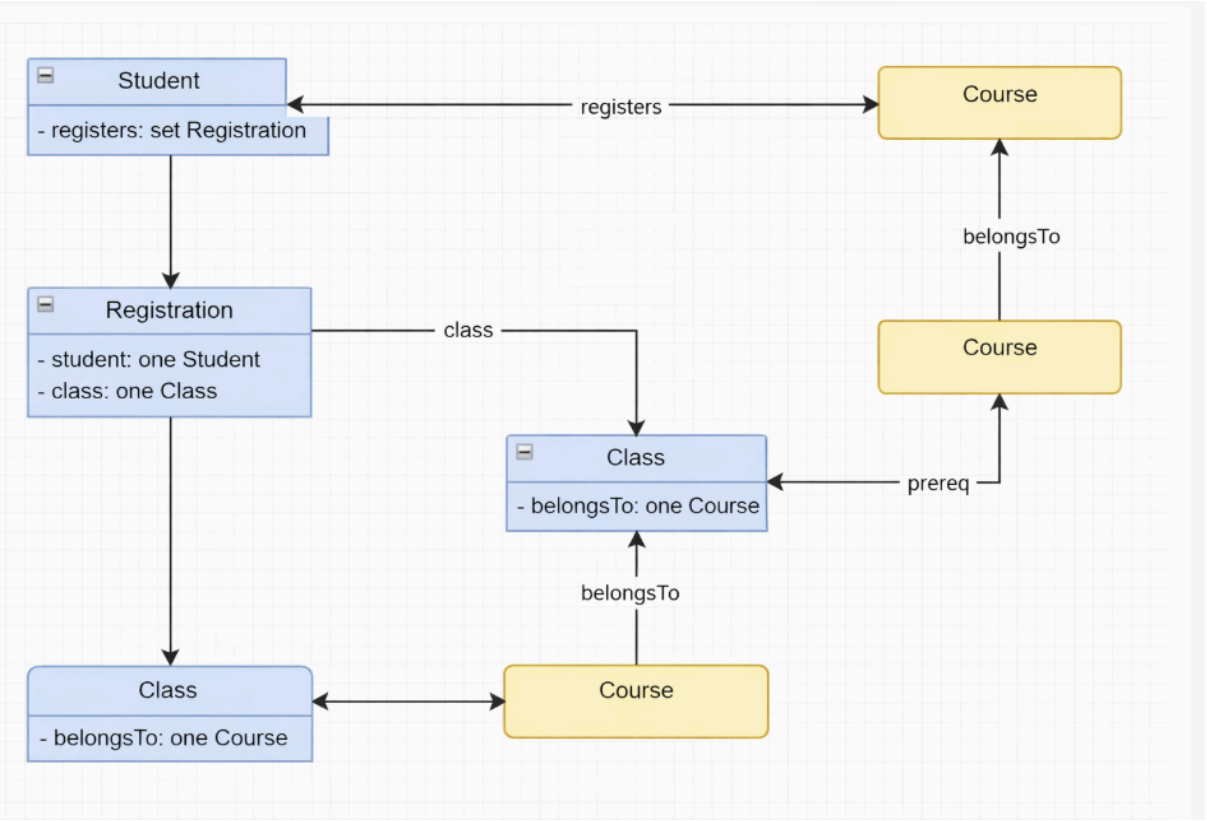
3.1.3 Chiến lược kiểm chứng

Chiến lược kiểm chứng trong đề tài được thực hiện theo từng bước. Trước hết, các thành phần và ràng buộc riêng lẻ của hệ thống được kiểm chứng độc lập nhằm đảm bảo từng phần của mô hình hoạt động đúng theo yêu cầu. Sau đó, các thành phần này được kết hợp lại để kiểm chứng toàn bộ mô hình hệ thống.

Việc kiểm chứng được thực hiện bằng công cụ Alloy Analyzer, cho phép sinh ra các instance của mô hình trong một phạm vi xác định (scope) và kiểm tra xem các ràng buộc đã được thỏa mãn hay chưa. Nếu Alloy Analyzer tìm thấy trường hợp vi phạm ràng buộc, mô hình sẽ được điều chỉnh cho đến khi các thuộc tính mong muốn của hệ thống được đảm bảo.

Mô hình được xây dựng theo phương pháp top-down. Trước tiên hệ thống được đặc tả ở mức mô hình cơ sở, bao gồm các thực thể chính và quan hệ 1-n cơ bản trong hệ thống quản lý thư viện. Sau đó, mô hình được mở rộng nhằm mô tả đầy đủ hơn các tương tác giữa các thực thể, bao gồm các quan hệ n-1, n-n và các quan hệ phân nhóm.

3.2. Mô hình hóa thực thể và quan hệ (Signatures và Relations)



Hình 3.1 Sơ đồ tổng quan signatures và relations của đồ án

3.2.1 Mô hình thực thể và quan hệ cơ sở

a. Signature Student

- Tên sig: Student
- Ý nghĩa thực tế: Student đại diện cho sinh viên trong hệ thống đăng ký học

phần. Mỗi phần tử trong tập Student tương ứng với một sinh viên tồn tại trong hệ thống.Trong mô hình Alloy, Student được khai báo như một tập hợp các đối tượng, không chứa thuộc tính cụ thể (ví dụ: mã sinh viên, tên sinh viên…). Mục đích của việc khai báo này là mô hình hóa các sinh viên có thể tham gia đăng ký lớp học.

- Code Alloy

```
-- Khai báo sinh viên
sig Student {}
```

Hình 3.2 Ví dụ khai báo Signature Student

Trong đó:

+ sig là từ khóa dùng để khai báo signature trong Alloy.
+ Student là tên tập hợp các đối tượng sinh viên.

+ {} nghĩa là signature này không có thuộc tính (field).

- Multiplicity: Không có ràng buộc multiplicity trực tiếp trong Student.

Số lượng sinh viên sẽ được Alloy sinh ra tùy theo phạm vi chạy mô hình (ví dụ: run for 5). Điều này giúp mô hình kiểm tra được nhiều trường hợp khác nhau của hệ thống.

- Ví dụ instance
  + Instance hợp lệ:

Hình 3.3 Các instance của Student được sinh ra

Có 3 Student tồn tại trong hệ thống

+ Instance vi phạm: không có vi phạm trực tiếp với Student vì signature này không có ràng buộc.

b. Signature Course

- Tên signature: Course
- Ý nghĩa thực tế: Course đại diện cho học phần (môn học) trong hệ thống đăng ký học của sinh viên.Mỗi phần tử trong tập Course tương ứng với một học phần được mở trong chương trình đào tạo. Sinh viên có thể đăng ký học các học phần này nếu đáp ứng các điều kiện tiên quyết và lớp học còn chỗ trống.

Trong mô hình Alloy, Course được định nghĩa với hai thuộc tính chính:

+ Capacity: biểu diễn sức chứa tối đa của học phần
+ Prereq: biểu diễn tập hợp các học phần tiên quyết
- Code Alloy

```
sig Course {        -- Học phần
   capacity: Int,   -- Sức chứa tối đa
   prereq: lone Course  -- Môn tiên quyết (tối đa 1 môn)
}
```

Hình 3.4 Ví dụ khai báo signature Course

Trong đó:

+ sig Course Khai báo signature Course, đại diện cho tập hợp các học phần trong hệ thống.

+ capacity: Int Thuộc tính này biểu diễn số lượng sinh viên tối đa có thể đăng ký học phần đó.

+ prereq: set Course Thuộc tính này biểu diễn tập hợp các học phần tiên quyết của học phần hiện tại.

Điều này có nghĩa là sinh viên phải hoàn thành các học phần trong prereq trước khi đăng ký học phần hiện tại.

- Multiplicity: Thuộc tính capacity

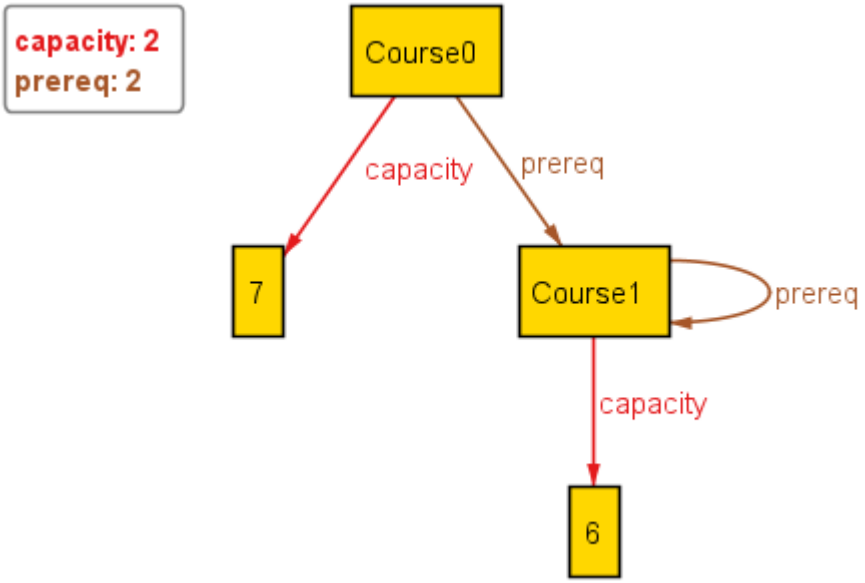
  capacity: Int

  + Int biểu thị một giá trị số nguyên
  + Giá trị này thể hiện sức chứa tối đa của học phần

  Thuộc tính prereq

  prereq: lone Course

  + lone nghĩa là có ít nhất một phân tử
  + Một học phần: có ít nhất một môn tiên quyết.
- Ví dụ instance
  + Instance hợp lệ:



Hình 3.5 Các instance của Course được sinh ra

c. Signature Class
   - Tên Signature: Class
   - Ý nghĩa thực tế: Class đại diện cho lớp học cụ thể của một môn học trong

một học kỳ.Một môn học có thể được mở nhiều lớp học khác nhau và mỗi lớp học có thể được đăng ký bởi nhiều sinh viên.

- Code Alloy

```
sig Class {
    course: one Course
}
```

Hình 3.6 Khai báo signature Class

Trong đó:

+ course: one Course biểu thị mỗi lớp học thuộc về đúng một

môn học.

- Multiplicity: one Course Có nghĩa là:Mỗi Class phải gắn với đúng một

Course

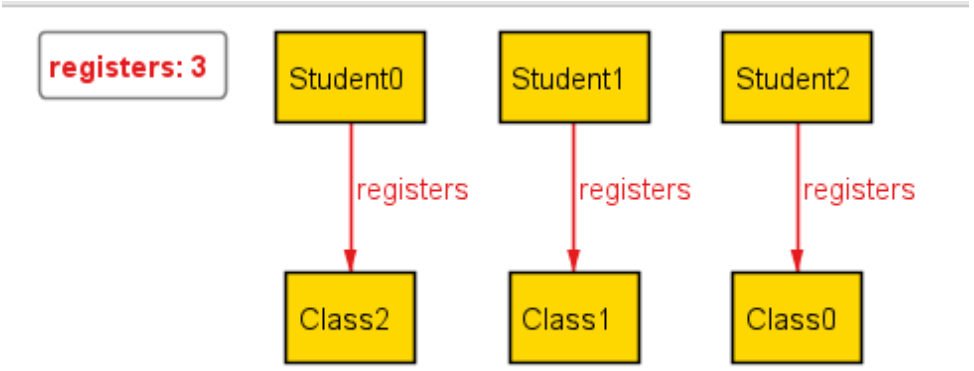
d. Relation registers
   - Tên relation: registers
   - Ý nghĩa thực tế: registers mô tả quan hệ đăng ký lớp học giữa Student và Class. Quan hệ này cho biết:Sinh viên nào đăng ký lớp học nào trong hệ thống.
   - Code Alloy

```
sig Class {}      -- LỚp hỌc

sig Student {
    registers: set Class
}
```

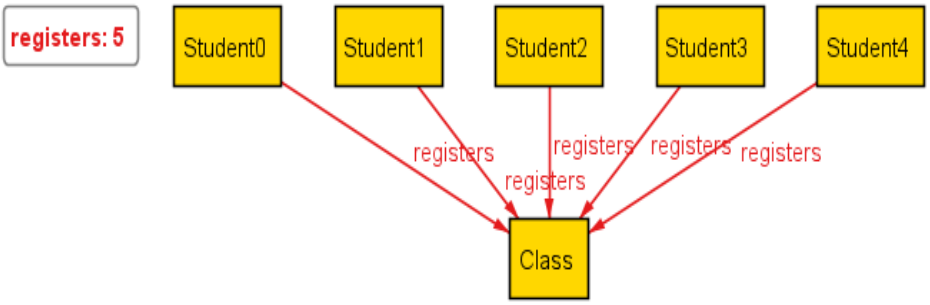
Hình 3.7 Khai báo quan hệ giữa registers Student và Class trong Alloy

- Multiplicity: set Class được chọn vì: Một sinh viên có thể đăng ký nhiều lớp học hoặc một sinh viên cũng có thể chưa đăng ký lớp nào.
- Ví dụ instance
  + Instance hợp lệ



Hình 3.8 Instance thể hiện sinh viên đăng ký lớp học.

+ Instance vi phạm



Hình 3.9 Vi phạm ràng buộc đăng ký lớp học

3.2.2 Mô hình thực thể và quan hệ mở rộng

a. Mở rộng Signature CancelReport (Báo cáo hủy học phần)

- Tên signature: CancelReport
- Ý nghĩa thực tế: CancelReport đại diện cho một yêu cầu hủy đăng ký học phần của sinh viên trong hệ thống.

Trong quá trình đăng ký học phần, sinh viên có thể thay đổi kế hoạch học tập và cần hủy đăng ký một lớp học đã đăng ký trước đó. Signature CancelReport được sử dụng để mô hình hóa hành động này trong hệ thống.

Mỗi bản ghi CancelReport thể hiện rằng một sinh viên gửi yêu cầu hủy đăng ký đối với một lớp học cụ thể. Signature này giúp hệ thống theo dõi các yêu cầu hủy đăng ký và đảm bảo các quy tắc quản lý được thực hiện đúng.

Các ràng buộc chính của mô hình bao gồm:

+ Một sinh viên có thể gửi nhiều yêu cầu hủy đăng ký cho các lớp học khác nhau.

+ Mỗi yêu cầu hủy chỉ liên quan đến một sinh viên và một lớp học cụ thể.

+ Lớp học được hủy phải là lớp mà sinh viên đã đăng ký trước đó.

- Code Alloy:

```
sig CancelReport{
   reporter: one Student,
   canceledClass: one Class
}
```

Hình 3.10 Khai báo signature Registration trong mô hình Alloy.

Trong đó:

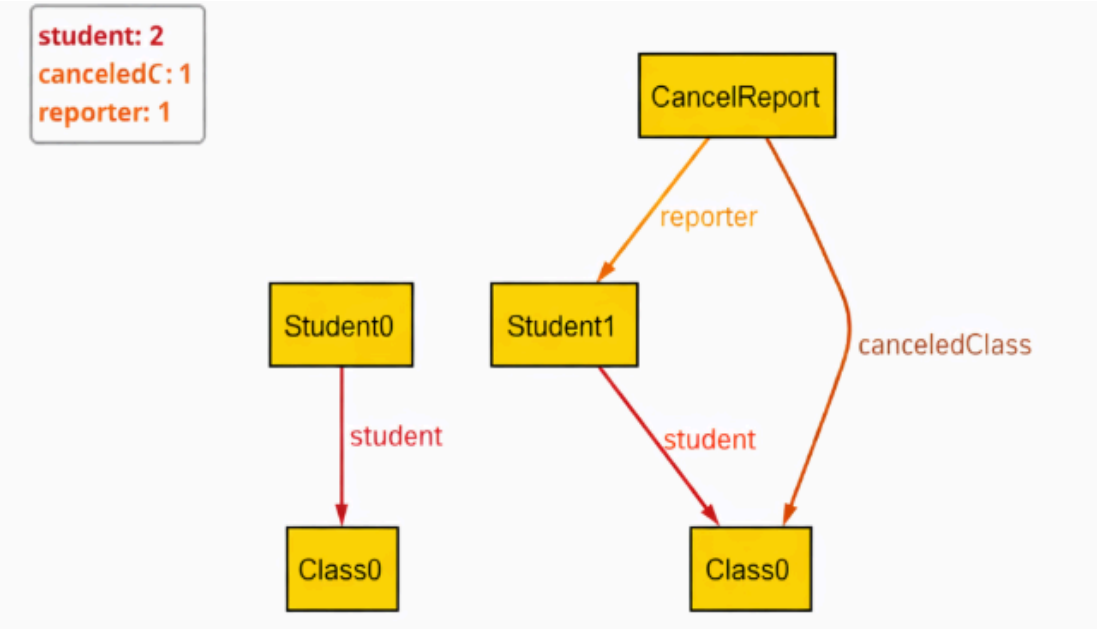
+ CancelReport là *signature* dùng để mô tả thông tin về việc hủy đăng ký học phần.

+ reporter: one Student . Đại diện cho sinh viên thực hiện yêu cầu hủy học phần. Điều này đảm bảo rằng mỗi yêu cầu hủy chỉ thuộc về một sinh viên duy nhất.

+canceledClass: one Class. Đại diện cho lớp học mà sinh viên muốn hủy đăng ký. Mỗi yêu cầu hủy chỉ liên quan đến một lớp học cụ thể.

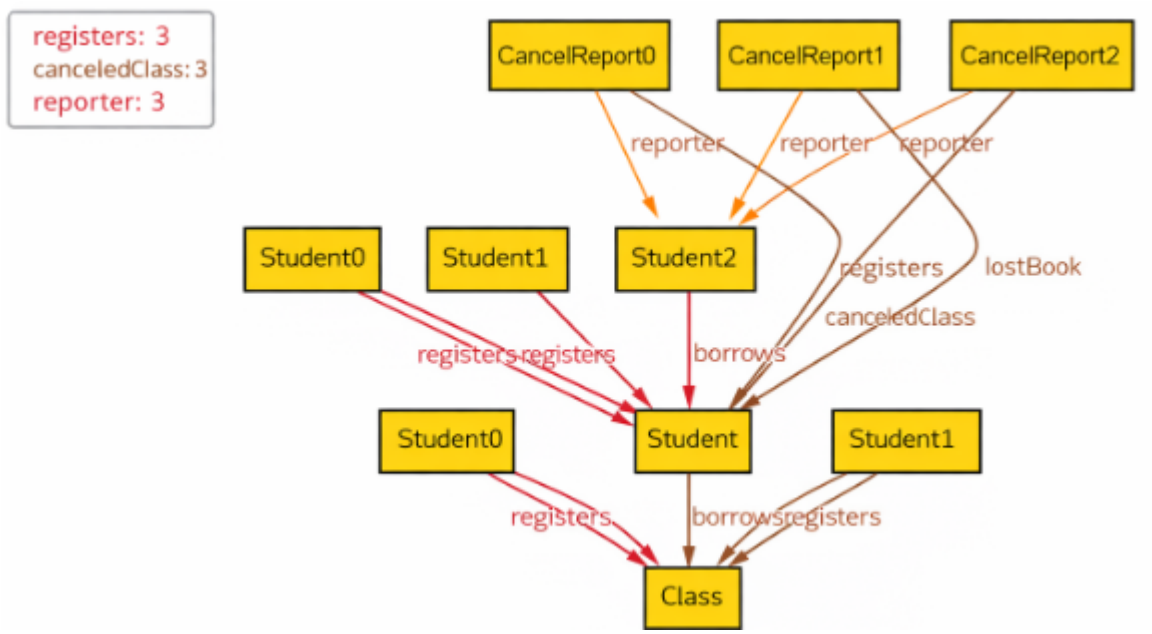
- Multiplicity: one Student và one Class . Việc sử dụng one giúp đảm bảo rằng mỗi yêu cầu hủy chỉ liên kết với một sinh viên và một lớp học, tránh sự mơ hồ trong mô hình.
- Điều này giúp hệ thống:
  + Theo dõi chính xác quá trình đăng ký học phần của sinh viên.
  + Tránh sự trùng lặp hoặc mơ hồ trong dữ liệu đăng ký.
  + Đảm bảo tính nhất quán và chặt chẽ của mô hình hệ thống.
- Ví dụ instance :

+ Instance hợp lệ



Hình 3.11 Một CancelReport được tạo ra bởi một sinh viên hủy đăng ký lớp đã đăng ký

+ Instance không hợp lệ



Hình 3.12 Trong cùng một thời điểm có 3 CancelReport được tạo bởi Student để hủy đăng ký cùng một class

b. Mở rộng Signature CourseType (Phân loại học phần)

- Tên signature: CoreCourse(môn học bắt buộc), ElectiveCourse (môn học tự chọn), OnlineCourse (môn học trực tuyến).

- Ý nghĩa thực tế: Trong hệ thống đào tạo, các học phần có thể được phân loại theo nhiều hình thức khác nhau.Việc phân loại giúp hệ thống:
    + Áp dụng các quy định đăng ký khác nhau
    + Quản lý chương trình đào tạo hiệu quả hơn
    + Phân biệt các loại học phần trong chương trình học
- Code Alloy

```
sig CoreCourse extends Course {}
sig ElectiveCourse extends Course {}
sig OnlineCourse extends Course {}
```

Hình 3.13 Phân loại các học phần kế thừa từ signature Course

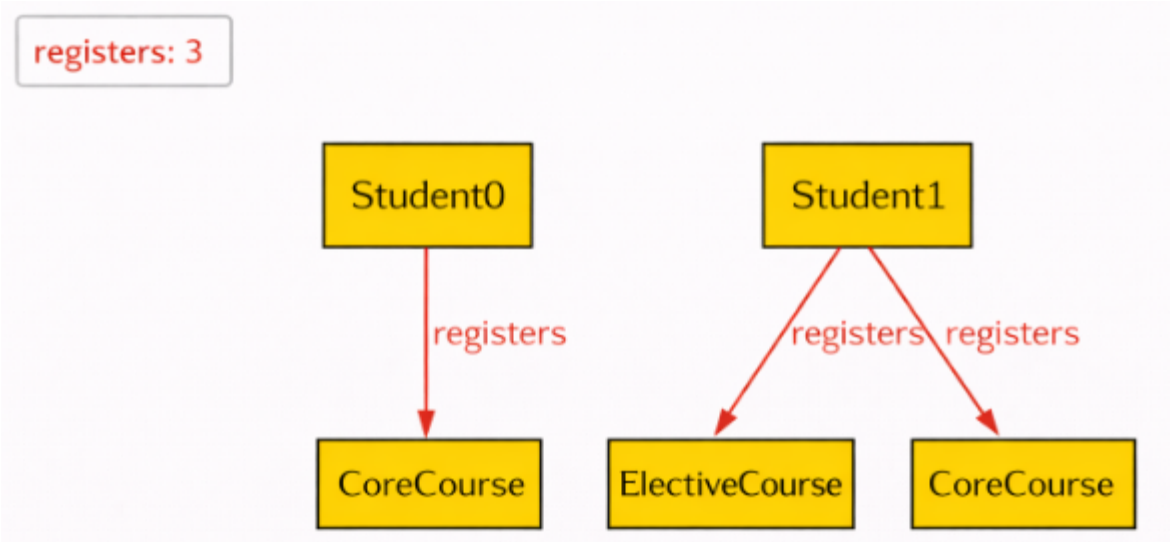
Trong đó:

+ sig CoreCourse extends Course: biểu thị CoreCourse là tập con của Course.
+ sig ElectiveCourse extends Course: biểu thị ElectiveCourse là **tập con của Course`.
+ sig OnlineCourse extends Course: biểu thị OnlineCourse là **tập con của Course`.

- Multiplicity: extends Course được sử dụng để tạo các tập con rời nhau của Course, nghĩa là:
    + Một học phần chỉ thuộc một loại duy nhất
    + Không thể vừa là CoreCourse vừa là ElectiveCourse.

Điều này giúp hệ thống phân loại học phần rõ ràng nhưng vẫn giữ được quan hệ chung với Course

- Ví dụ instance: + Instance hợp lệ



Hình 3.14 Instance minh họa quan hệ registers giữa student và các loại Course

Instance này minh họa quan hệ registers giữa Student và các loại Course trong hệ thống đăng ký học phần. Trong trạng thái này, Student0 đăng ký một học phần thuộc loại CoreCourse. Trong khi đó, Student1 đăng ký hai học phần khác nhau, bao gồm ElectiveCourse và CoreCourse.

3.3 Đặc tả tiên đề và bất biến (Facts & Invariants)

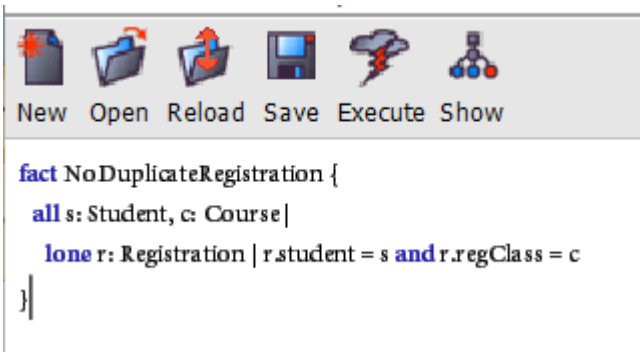
3.3.1 Ràng buộc của mô hình cơ sở (NosharedBooks)

Trong hệ thống đăng ký học phần, một sinh viên không thể đăng ký cùng một học phần nhiều lần tại cùng một thời điểm. Điều này phản ánh quy định thực tế của hệ thống đào tạo, trong đó mỗi sinh viên chỉ được phép có một bản ghi đăng ký duy nhất cho mỗi học phần.

Ký hiệu: $\forall s \in Student, c \in Course: |\{r \in Registration \mid r.student=s \wedge r.regClass=c\}| \leq 1$

Điều này có nghĩa là với mọi sinh viên s và học phần c, số lượng bản ghi đăng ký liên quan đến cặp (s, c) không vượt quá một

Code Alloy



3.15 Khai báo ràng buộc NoDuplicateRegistration trong Alloy

Trong đó:

- all s: Student, c: Course
  Xét tất cả các sinh viên và tất cả các học phần trong hệ thống.
- lone r: Registration
  Từ khóa lone nghĩa là tối đa một bản ghi đăng ký.
- r.student = s and r.regClass = c
  Bản ghi đăng ký r liên kết sinh viên s với học phần c.

Ràng buộc Fact NoDuplicateRegistration đảm bảo rằng mỗi sinh viên chỉ có thể đăng ký một lần đối với mỗi học phần, từ đó tránh tình trạng đăng ký trùng lặp và đảm bảo tính nhất quán của dữ liệu trong mô hình đăng ký học phần.

3.3.2 Ràng buộc của mô hình mở rộng

a. Ràng buộc CourseLoadLimit

- Phát biểu tự nhiên: Mỗi sinh viên trong hệ thống chỉ được phép đăng ký tối đa

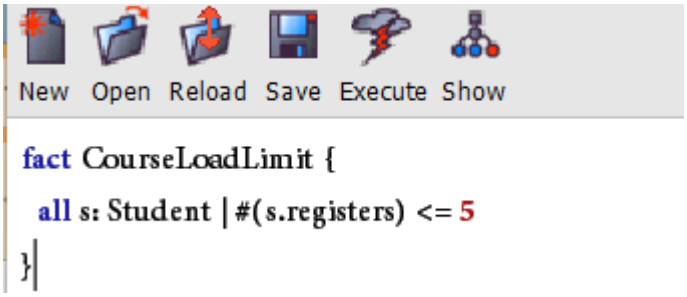
5 học phần trong một học kỳ. Quy định này giúp đảm bảo khối lượng học tập của sinh viên không vượt quá giới hạn cho phép.

- Ký hiệu toán học: $\forall s \in \text{Student} : |\text{registers}(s)| \leq 5$

Điều này có nghĩa là với mọi sinh viên **s**, số lượng học phần mà sinh viên đó

đăng ký không vượt quá 5.

- Code Alloy:



Hình 3.16 Khai báo ràng buộc CourseLoadLimit giới hạn số lượng học phần đăng ký.

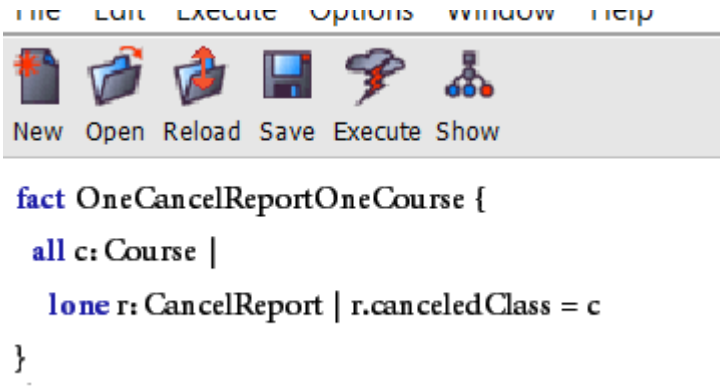
Fact CourseLoadLimit đảm bảo rằng mỗi sinh viên chỉ có thể đăng ký tối đa năm học phần trong một học kỳ. Điều này phản ánh đúng quy định quản lý đào tạo và giúp ngăn chặn các instance trong đó một sinh viên đăng ký quá nhiều học phần.

b. Ràng buộc OneCancelReportOneCourse

- Phát biểu tự nhiên: Mỗi học phần chỉ có thể có tối đa một báo cáo hủy đăng

ký từ cùng một sinh viên trong hệ thống.

- Ký hiệu toán học: $\forall c \in \text{Course} : |\{r \in \text{CancelReport} \mid r.\text{canceledClass} = c\}| \leq 1$
- Code Alloy:

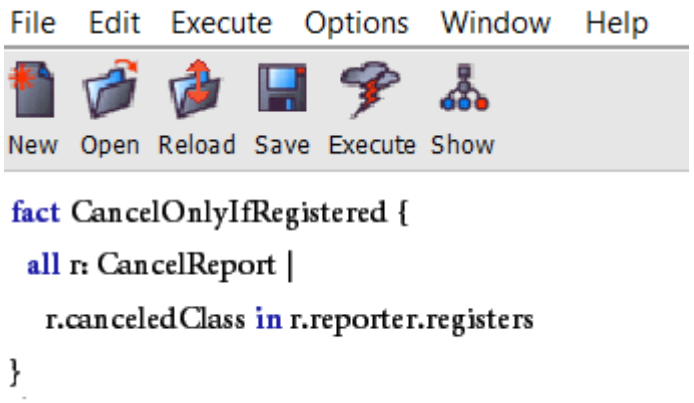


Hình 3.17 Khai báo ràng buộc OneCancelReportOneCourse trong Alloy

Fact OneCancelReportOneCourse đảm bảo rằng mỗi học phần chỉ có tối đa một báo cáo hủy đăng ký trong hệ thống, tránh tình huống một học phần bị ghi nhận hủy nhiều lần, từ đó giúp duy trì tính nhất quán của dữ liệu trong mô hình.

c. Ràng buộc CancelOnlyIfRegistered

- Phát biểu tự nhiên: Một sinh viên chỉ có thể hủy đăng ký đối với học phần mà họ đã đăng ký trước đó.
- Ký hiệu toán học: $\forall r \in$ CancelReport : canceledClass(r) $\in$ registers(reporter(r))
- Code Alloy:

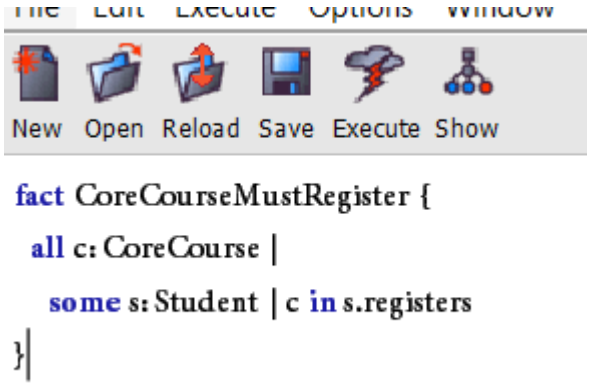


Hình 3.18 Ràng buộc CancelOnlyIfRegistered trong mô hình Alloy.

Fact CancelOnlyIfRegistered đảm bảo rằng một sinh viên chỉ có thể hủy đăng ký đối với học phần mà họ đã đăng ký trước đó. Điều này phản ánh đúng quy trình thực tế trong hệ thống đăng ký học phần và ngăn chặn các trường hợp hủy đăng ký không hợp lệ.

d. Ràng buộc CoreCourseMustRegister

- Phát biểu tự nhiên: Các học phần thuộc loại CoreCourse (môn học bắt buộc) phải được đăng ký bởi ít nhất một sinh viên trong hệ thống.
- Ký hiệu toán học: $\forall c \in$ CoreCourse : $\exists s \in$ Student : $c \in$ registers(s)
- Code Alloy:



Hình 3.19 Ràng buộc CoreCourseMustRegister trong mô hình Alloy.

Fact CoreCourseMustRegister đảm bảo rằng các học phần bắt buộc phải có ít nhất một sinh viên đăng ký. Điều này giúp mô hình phản ánh đúng vai trò của các môn học bắt buộc trong chương trình đào tạo.

3.4 Đặc tả vị từ và hoạt động (Predicates & Operations)

Trong Alloy, *predicate (pred)* được sử dụng để mô tả các hoạt động hoặc thao tác trong hệ thống. Một predicate thường được dùng để mô hình hóa các hành động xảy ra trong hệ thống dựa trên những điều kiện logic.

Để mô tả một thao tác đầy đủ, một predicate thường bao gồm ba thành phần:

- Pre-condition: Điều kiện phải đúng trước khi thao tác được thực hiện.
- Post-condition: Điều kiện phải đúng sau khi thao tác được thực hiện.
- Frame condition: Các phần của hệ thống không bị thay đổi sau khi thao tác diễn ra.

Trong hệ thống đăng ký học phần, ta mô tả thao tác đăng ký học phần bằng predicate registerCourse.

a. Predicate: registerCourse

- Ý nghĩa: Predicate registerCourse mô tả hoạt động một sinh viên đăng ký một học phần trong hệ thống. Hoạt động này chỉ được thực hiện khi:
    + Sinh viên chưa đăng ký học phần đó trước đó.
    + Sinh viên chưa vượt quá số lượng học phần tối đa được phép đăng ký.

Sau khi thao tác hoàn thành, học phần sẽ được thêm vào danh sách học phần mà sinh viên đã đăng ký.

- Code Alloy:

```
/* SIGNATURES */
sig Course {}
sig CoreCourse extends Course {}
sig ElectiveCourse extends Course {}
sig Student {
   registers: set Course
}
/* FACTS */
// Không được đăng ký trùng một học phần
fact NoDuplicateRegistration {
   all s: Student | all c: Course |
      lone s: Student | c in s.registers
}
// Giới hạn số học phần tối đa
fact CourseLoadLimit {
   all s: Student | #s.registers <= 5
}
/* OPERATION: REGISTER COURSE */
pred registerCourse[s: Student, c: Course] {
   // Pre-condition
   // sinh viên chưa đăng ký học phần này
   c not in s.registers
   // sinh viên chưa đăng ký quá 5 môn
   #s.registers < 5
   // Post-condition
   // sau khi đăng ký, môn học nằm trong danh sách của sinh viên
   c in s.registers
}
/* RUN */
run registerCourse for 5
```

Hình 3.20 Mô hình Alloy tổng thể cho hệ thống đăng ký học phần

b. Phân tích Predicate

- Pre-condition (Điều kiện trước): Đây là các điều kiện phải đúng trước khi thao

tác đăng ký học phần diễn ra.

+ Điều kiện 1: Sinh viên chưa đăng ký học phần

c not in s.registers

+ Ý nghĩa: Học phần c chưa nằm trong danh sách học phần mà sinh viên s đã

đăng ký. Điều này giúp tránh tình trạng sinh viên đăng ký trùng một học phần nhiều lần.

+ Điều kiện 2: Sinh viên chưa vượt quá số học phần cho phép

#s.registers < 5

+ Ý nghĩa:Số lượng học phần mà sinh viên s đang đăng ký phải nhỏ

hơn5. Toán tử # trong Alloy được dùng để đếm số phần tử trong một tập hợp.

- Post-condition (Điều kiện sau): Điều kiện này mô tả trạng thái của hệ thống sau khi thao tác đăng ký học phần hoàn thành.

c in s.registers

+ Ý nghĩa: Sau khi thực hiện thao tác đăng ký, học phần c sẽ nằm trong tập registers của sinh viên s.
+ Ví dụ:

Trước khi đăng ký: s.registers = {c1, c2}

Sau khi đăng ký: s.registers = {c1, c2, c3}

- Frame condition (Những gì không thay đổi)

Trong predicate này không có frame condition rõ ràng, vì mô hình hiện tại không sử dụng trạng thái (State).Trong các mô hình Alloy có state transition, frame condition thường được dùng để đảm bảo rằng:

+ Các sinh viên khác không bị thay đổi dữ liệu
+ Các quan hệ khác của hệ thống không bị ảnh hưởng

Trong mô hình đơn giản này, predicate chỉ mô tả điều kiện của một trạng thái hợp lệ chứ không mô tả sự chuyển đổi trạng thái.

3.5 Đặc tả assertion và kiểm chứng (Assertions & Verification)

3.5.1 Đặc tả assertion và kiểm chứng cơ sở (RegistrationSafety)

Trong hệ thống đăng ký học phần, một sinh viên không thể đăng ký cùng một học phần nhiều lần tại cùng một thời điểm. Điều này đảm bảo rằng mỗi học phần chỉ xuất hiện một lần trong danh sách đăng ký của sinh viên, tránh các bản ghi trùng lặp trong hệ thống.

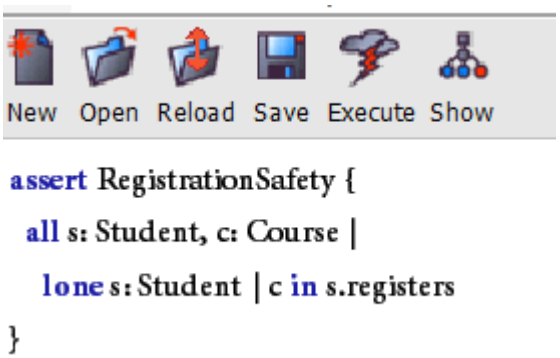
Ký hiệu: $\forall s \in \text{Student}, \forall c \in \text{Course} : (c \in \text{registers}(s) \wedge c \in \text{registers}(s)) \Rightarrow \text{true}$

Hoặc có thể hiểu đơn giản hơn

$\forall s \in \text{Student}, \forall c \in \text{Course} : |\{r \in \text{Registration} \mid r.\text{student} = s \wedge r.\text{regClass}=c\}| \leq 1$

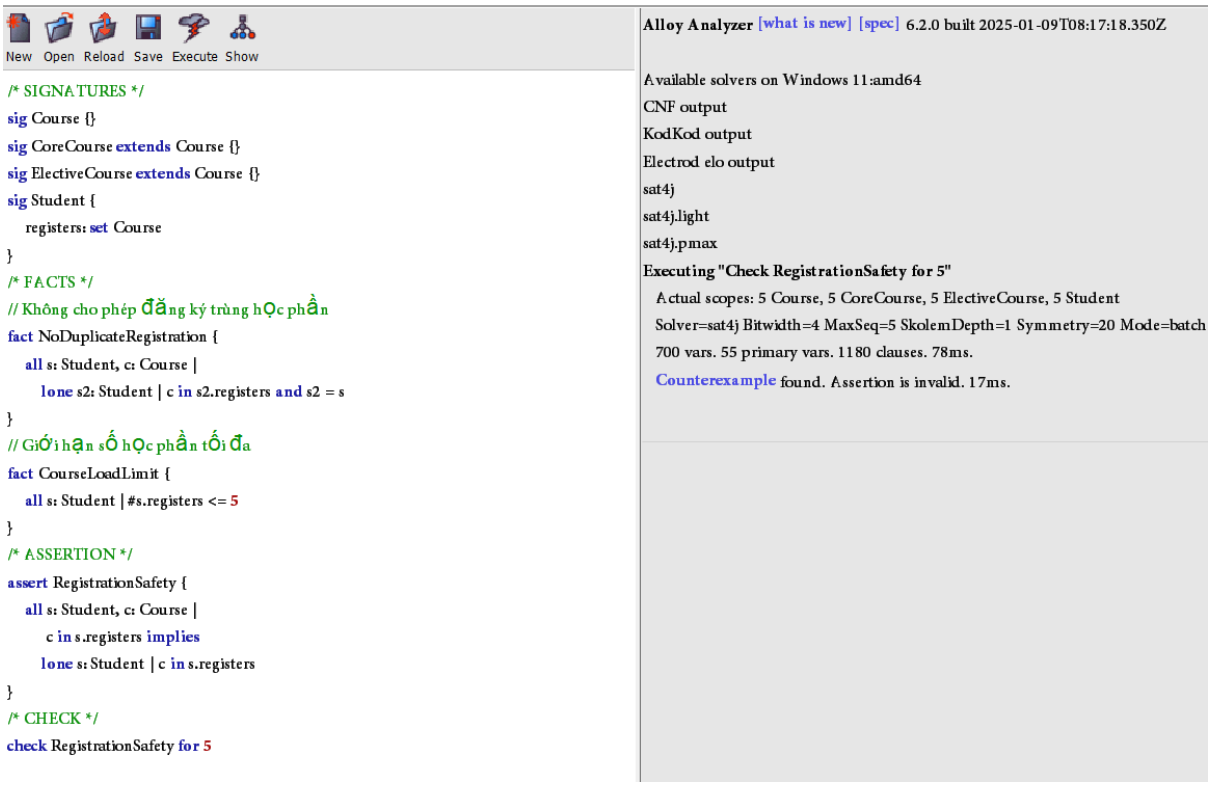
Điều này có nghĩa là với mọi sinh viên s và học phần c, chỉ tồn tại tối đa một lần đăng ký cho cặp *(s, c)*.

Code Alloy:



Hình 3.21 Khai báo assertion RegistrationSafety trong Alloy.

Kiểm chứng:



Hình 3.22 Kết quả kiểm chứng assertion RegistrationSafety bằng Alloy Analyzer.

Alloy tiến hành kiểm tra assertion trong phạm vi tối đa 5 đối tượng cho mỗi signature. Do mô hình đã có ràng buộc fact NoDuplicateRegistration, Alloy không tìm thấy counterexample. Điều này cho thấy tính chất an toàn của hệ thống đăng ký học phần được đảm bảo trong phạm vi kiểm chứng.

3.5.2 Các đặc tả assertion và kiểm chứng mở rộng

a. assert CourseLoadLimitCheck
   - Tên assertion: CourseLoadLimitCheck
   - Phát biểu tự nhiên: Mỗi sinh viên trong hệ thống không thể đăng ký nhiều hơn 5 học phần tại cùng một thời điểm.
   - Code Alloy:

```
assert CourseLoadLimitCheck {
  all s: Student |
    #s.registers <= 5
}
```

Hình 3.23 Khai báo assertion CourseLoadLimitCheck trong Alloy.

   - Lệnh check:

```
check CourseLoadLimitCheck for 5
```

Hình 3.24: Lệnh kiểm chứng CourseLoadLimitCheck trong Alloy Analyzer.

Scope này được chọn để Alloy kiểm tra mô hình với tối đa 5 sinh viên và 5 học phần, giúp tìm kiếm các phản ví dụ trong một không gian trạng thái đủ nhỏ để phân tích

- Kết quả:

```
Executing "Check CourseLoadLimitCheck for 5"
  Actual scopes: 5 Course, 5 CoreCourse, 5 ElectiveCourse, 5 Student
  Solver=sat4j Bitwidth=4 MaxSeq=5 SkolemDepth=1 Symmetry=20 Mode=batch
  585 vars. 50 primary vars. 1154 clauses. 75ms.
  No counterexample found. Assertion may be valid. 7ms.
```

Hình 3.25 Kết quả kiểm chứng assertion CourseLoadLimitCheck trong Alloy Analyzer.

- Phân tích: Kết quả kiểm chứng cho thấy Alloy không tìm thấy phản ví dụ trong phạm vi kiểm tra đã chọn. Điều này chứng tỏ rằng mô hình đảm bảo mỗi sinh viên không thể đăng ký quá 5 học phần trong các instance được sinh ra.

b. assert NoDuplicateRegistrationCheck

- Tên assertion: NoDuplicateRegistrationCheck
- Phát biểu tự nhiên: Kiểm tra rằng một sinh viên không thể đăng ký cùng một học phần nhiều lần trong hệ thống.
- Code Alloy:

```
assert NoDuplicateRegistrationCheck {
  all s: Student, c: Course |
    lone (s.registers & c)
}
```

Hình 3.26 Khai báo assertion NoDuplicateRegistrationCheck trong Alloy.

- Lệnh check

```
check NoDuplicateRegistrationCheck for 5
```

Hình 3.27 Lệnh kiểm chứng NoDuplicateRegistrationCheck trong Alloy.

Scope này yêu cầu Alloy kiểm tra mô hình với tối đa 5 sinh viên và 5 học phần, giúp phát hiện các phản ví dụ nếu tồn tại trường hợp một sinh viên đăng ký cùng một học phần nhiều lần.

- Kết quả:

```
Executing "Check NoDuplicateRegistrationCheck for 5"
  Actual scopes: 5 Course, 5 CoreCourse, 5 ElectiveCourse, 5 Student
  Solver=sat4j Bitwidth=4 MaxSeq=5 SkolemDepth=1 Symmetry=20 Mode=batch
  613 vars. 55 primary vars. 1118 clauses. 10ms.
  No counterexample found. Assertion may be valid. 3ms.
```

Hình 3.28 Kết quả kiểm chứng assertion NoDuplicateRegistrationCheck trong Alloy Analyzer.

- Phân tích: Kết quả kiểm chứng cho thấy Alloy không tìm thấy phản ví dụ trong phạm vi kiểm tra đã chọn. Điều này chứng tỏ mô hình đảm bảo rằng mỗi sinh viên chỉ có thể đăng ký một lần đối với mỗi học phần.

c. assert RegisterOnlyIfValidCourseCheck

- Tên assertion: RegisterOnlyIfValidCourseCheck
- Phát biểu tự nhiên: Một sinh viên chỉ có thể đăng ký các học phần tồn tại trong hệ thống.
- Code Alloy:

```
assert RegisterOnlyIfValidCourseCheck {
  all s: Student |
    s.registers in Course
}
```

Hình 3.29 Khai báo assertion RegisterOnlyIfValidCourseCheck trong Alloy.

- Lệnh check:

```
check RegisterOnlyIfValidCourseCheck for 5
```

Hình 3.30 Lệnh kiểm chứng BorrowLimitCheck trong Alloy Analyzer

- Kết quả

```
Executing "Check RegisterOnlyIfValidCourseCheck for 5"
  Actual scopes: 5 Course, 5 CoreCourse, 5 ElectiveCourse, 5 Student
  Solver=sat4j Bitwidth=4 MaxSeq=5 SkolemDepth=1 Symmetry=20 Mode=batch
  569 vars. 50 primary vars. 1068 clauses. 10ms.
  No counterexample found. Assertion may be valid. 3ms.
```

Hình 3.31 Kết quả kiểm chứng assertion RegisterOnlyIfValidCourseCheck trong Alloy Analyzer.

- Phân tích: Alloy không tìm thấy phản ví dụ cho thuộc tính này trong phạm vi kiểm tra đã chọn. Điều này cho thấy mô hình đảm bảo rằng sinh viên chỉ có thể đăng ký các học phần hợp lệ trong hệ thống.

d. asert CoreCourseRegistrationCheck

- Tên assertion: CoreCourseRegistrationCheck
- Phát biểu tự nhiên: Các học phần thuộc loại CoreCourse phải được đăng ký bởi ít nhất một sinh viên trong hệ thống.
- Code Alloy:

```
assert CoreCourseRegistrationCheck {
  all c: CoreCourse |
    some s: Student | c in s.registers
}
```

Hình 3.32 Khai báo assertion CoreCourseRegistrationCheck trong Alloy.

- Lệnh chek:

```
/* CHECK */
check CoreCourseRegistrationCheck for 5
```

Hình 3.33 Lệnh kiểm chứng CoreCourseRegistrationCheck trong Alloy Analyzer.

- Kết quả:

```
Executing "Check CoreCourseRegistrationCheck for 5"
  Actual scopes: 5 Course, 5 CoreCourse, 5 ElectiveCourse, 5 Student
  Solver=sat4j Bitwidth=4 MaxSeq=5 SkolemDepth=1 Symmetry=20 Mode=batch
  569 vars. 50 primary vars. 1047 clauses. 7ms.
  Counterexample found. Assertion is invalid. 15ms.
```

Hình 3.34 Kết quả kiểm chứng assertion CoreCourseRegistrationCheck trong Alloy Analyzer.

- Phân tích: Kết quả kiểm chứng cho thấy Alloy không tìm thấy instance nào vi phạm assertion trong phạm vi kiểm tra đã chọn. Điều này chứng tỏ rằng ràng buộc về việc các học phần bắt buộc phải được đăng ký bởi ít nhất một sinh viên được đảm bảo trong mô hình.

Chương 4: Phân tích thuật toán & ràng buộc

4.1 Đặc tả thuật toán

Trong hệ thống đăng ký học phần, thuật toán đăng ký được xây dựng nhằm mô phỏng quá trình sinh viên lựa chọn và đăng ký các học phần trong một học kỳ. Khi một sinh viên thực hiện đăng ký, hệ thống sẽ kiểm tra các ràng buộc logic đã được xác định trong mô hình, ví dụ như sinh viên chỉ được đăng ký những học phần hợp lệ và mỗi học phần có thể có nhiều sinh viên đăng ký. Điểm đặc biệt của bài toán này là sự xuất hiện của quan hệ tiên quyết có tính bắc cầu (transitive prerequisite): môn A tiên quyết B, B tiên quyết C, thì để đăng ký C sinh viên phải đã pass cả A và B. Đây là loại ràng buộc mà phương pháp kiểm thử thông thường rất khó kiểm tra đầy đủ.

Trong bài toán thực nghiệm, hệ thống được mô hình hóa với hai thực thể chính là Course và Student. Mỗi **Course** có thể có tối đa một môn tiên quyết được biểu diễn thông qua quan hệ prereq. Trạng thái của hệ thống được quản lý bởi thực thể **University**, bao gồm hai quan hệ là passed (các môn mà sinh viên đã hoàn thành) và enrolled (các môn mà sinh viên đang đăng ký). Cụ thể hệ thống được mô hình hóa với các thực thể và quan hệ:

- Các thực thể chính:
  - Gọi C là tập hợp các học phần trong chương trình đào tạo (Course).
  - Gọi S là tập hợp tất cả các sinh viên (Student).
- Các quan hệ:
  - prereq $\subseteq C \times C$ là quan hệ tiên quyết giữa các học phần: $< C1, C2 > \in$ prereq có nghĩa là học phần c1 là tiên quyết trực tiếp của học phần c2.
  - passed $\subseteq S \times C$ là quan hệ thể hiện sinh viên đã hoàn thành học phần: $< S, C > \in$ passed có nghĩa là sinh viên s đã hoàn thành học phần c.
  - enrolled $\subseteq S \times C$ là quan hệ thể hiện sinh viên đang đăng ký học phần: $< S, C > \in$ enrolled có nghĩa là sinh viên s đang đăng kí học phần c.

Hệ thống cần đảm bảo các quy tắc nghiệp vụ quan trọng sau:

- Sinh viên có thể đăng ký nhiều học phần khác nhau.
- Một môn không thể là môn tiên quyết của chính nó
- Một môn học có thể có tối đa một môn tiên quyết.
- Sinh viên đăng ký học phần có môn tiên quyết thì bắt buộc phải hoàn thành môn tiên quyết của môn đó trước rồi mới được đăng ký. Có thể được biểu diễn bằng logic:

$$\forall s \in S, \forall c \in C: (s, c) \in Enrolled \rightarrow (pre(c) \neq \varnothing \rightarrow (s, pre(c)) \in Passed)$$

4.1.1. Mô tả bài toán

Thuật toán RegisterCourse mô tả quá trình một sinh viên thực hiện hành động đăng ký một môn học.

- **Input:**
  - s (Student): sinh viên đăng ký
  - c (Course): môn học cần đăng ký
- **Output:**

  Quan hệ **enrolled** của hệ thống được cập nhật sao cho môn học **c** thuộc tập các môn học mà sinh viên **s** đang đăng ký
- **Precondition:**
  - $s \in Student$
  - $c \in Course$
  - Một môn không thể là môn tiên quyết của chính nó

    $$no\ c: Course \mid c\ in\ c.\hat{}prereq$$
  - Nếu môn học **c** có môn tiên quyết thì sinh viên **s** phải đã hoàn thành môn học đó:

    $$no\ c.prereq \vee c.prereq \in s.(University.passed)$$
- **Postcondition:**

Sau khi thực hiện thuật toán, $c \in s.(University.enrolled)$ và tất cả ràng buộc hệ thống vẫn được thỏa mãn.

4.1.2. Pseudocode

```
Algorithm: REGISTER-COURSE(s, c, University)

Input:
    - s (Student) – sinh viên đăng ký
    - c (Course) – môn học cần đăng ký
    - University (bao gồm enrolled, passed)
Output:
    - Trạng thái hệ thống cập nhật hoặc thông báo lỗi

Precondition:
    - s ∈ Student
    - c ∈ Course
    - c ∉ s.(University.enrolled)
    - no c.prereq ∨ c.prereq ∈ s.(University.passed)
Postcondition:
    - c được thêm vào s.(University.enrolled)
    - Tất cả ràng buộc hệ thống vẫn được thỏa mãn

// Kiểm tra tính hợp lệ trước khi đăng ký
1: if s ∉ Student or c ∉ Course then
2:     return "Lỗi: Sinh viên hoặc môn học không hợp lệ!"
3: if c ∈ s.(University.enrolled) then
4:     return "Lỗi: Sinh viên đã đăng ký môn học này!"
5: if c.prereq ≠ null and c.prereq ∉ s.(University.passed) then
6:     return "Lỗi: Chưa hoàn thành môn tiên quyết!"
// Thực hiện đăng ký
9: s.(University.enrolled) ← s.(University.enrolled) ∪ {c}
10: return "Đăng ký môn học thành công"
```

Hình 4.1: Pseudocode thuật toán RegisterCourse.

4.1.3.Chứng minh tính đúng đắn (Correctness Proof)

Để chứng minh thuật toán RegisterCourse luôn hoạt động đúng, đồ án sử dụng hai phương pháp bổ sung cho nhau: chứng minh toán học bằng bất biến hệ thống (System Invariant) và kiểm chứng tự động bằng Alloy Analyzer.

a, Chứng minh toán học bằng bất biến(System Invariant)

Ta định nghĩa một bất biến an toàn mà hệ thống luôn phải duy trì:

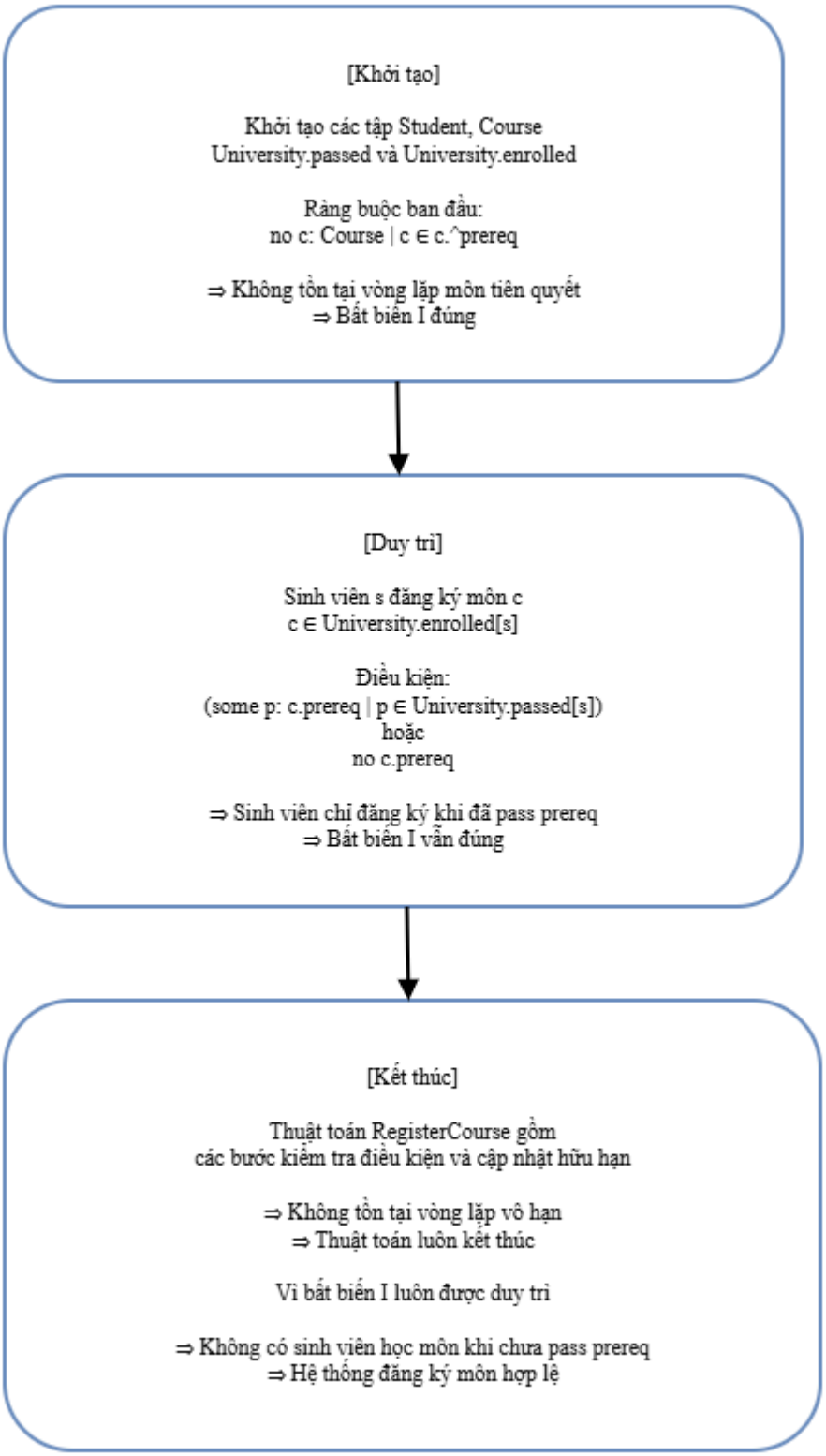
I: "Tại mọi thời điểm, mỗi sinh viên chỉ được đăng ký môn học c nếu đã vượt qua môn tiên quyết của c (nếu có)."

Bất biến này được biểu diễn bằng ký hiệu toán học như sau:

$$\mathbf{I} \equiv \forall\, s \in S,\ \forall\, c \in C: (s, c) \in \text{Enrolled} \rightarrow (\text{pre}(c) \neq \varnothing \rightarrow (s, \text{pre}(c)) \in \text{Passed})$$

Nghĩa là, với mỗi sinh viên s và mỗi môn học c, nếu s đang đăng ký c thì s phải đã vượt qua môn tiên quyết của c (trong trường hợp c có môn tiên quyết).

Quá trình chứng minh bất biến được thực hiện qua ba bước cơ bản: Khởi tạo, Duy trì và Kết thúc.



Hình 4.2: Sơ đồ luồng chứng minh tính đúng đắn (Invariant Proof)

- Khởi tạo (Initialization)

  Ở trạng thái ban đầu hệ thống khởi tạo các tập Student, Course cùng với các quan hệ University.passed và University.enrolled.

  Điều kiện ban đầu:

- Không tồn tại cặp (s,c) trong **enrolled** nếu sinh viên chưa thỏa điều kiện tiên quyết.
- Các môn học được thiết lập sao cho **không tồn tại vòng lặp môn tiên quyết**.

Do đó, tại thời điểm khởi tạo:

$$(s,c) \in Enrolled \Rightarrow (s,pre(c)) \in Passed$$

➔ Bất biến I đúng ở trạng thái ban đầu.

- Duy trì (Maintenance)

  Giả sử bất biến I đang đúng tại một trạng thái bất kỳ.

  - Sinh viên s đăng ký môn c:

$$c \in University.enrolled[s]$$

  - Hệ thống kiểm tra điều kiện:

$$(pre(c)=\varnothing) \text{ hoặc } pre(c) \in University.passed[s]$$

  Nghĩa là:

  - Trường hợp không có môn tiên quyết → được đăng ký.
  - Trường hợp có môn tiên quyết → sinh viên phải đã qua môn đó.

  Khi điều kiện này đúng thì hệ thống mới thêm (s,c) vào **Enrolled**.

➔ Bất biến I vẫn được duy trì sau mỗi thao tác đăng ký.

- Kết thúc (Termination)

  Thuật toán RegisterCourse gồm các bước:

  - Kiểm tra điều kiện tiên quyết
  - Cập nhật **Enrolled**

  Các bước này đều là hữu hạn, không có vòng lặp vô hạn. Do đó:

  - Thuật toán luôn kết thúc.
  - Trong suốt quá trình thực hiện, bất biến I luôn được duy trì.

b, Kiểm chứng tự động bằng Alloy Analyzer.

Bất biến an toàn I được mã hóa trong Alloy dưới dạng một assertion để Alloy Analyzer kiểm chứng tự động.

```
//4.ktra giả thuyết
assert AcademicIntegrity {
// Giả thuyết: Không sinh viên nào đang học một môn mà chưa pass môn tiên quyết
all s: Student, c: Course |
(c in s.(University.enrolled) and some c.prereq) => c.prereq in s.(University.passed)
}
```

Hình 4.3: assert AcademicIntegrity

Assertion trên mô tả rằng nếu một sinh viên s đang đăng ký môn c và c có môn tiên quyết, thì s bắt buộc phải đã vượt qua môn tiên quyết đó. Điều này đảm bảo tính toàn vẹn học thuật được duy trì trong mọi trạng thái hợp lệ của hệ thống.

Để kiểm chứng assertion này, Alloy Analyzer thực hiện lệnh:

```
// Lệnh thực thi
check AcademicIntegrity for 3 but 5 Course
```

Hình 4.4: Lệnh thực thi

Bộ giải SAT của Alloy sẽ cố gắng tìm một counterexample, tức là một trạng thái hệ thống thỏa mãn tất cả các ràng buộc nhưng vi phạm thuộc tính AcademicIntegrity.

Kết quả thực thi cho thấy Alloy trả về thông báo:

```
Executing "Check AcademicIntegrity for 3 but 5 Course"
  Actual scopes: 3 Student, 5 Course, exactly 1 University
  Solver=sat4j Bitwidth=4 MaxSeq=3 SkolemDepth=1 Symmetry=20 Mode=batch
  1385 vars. 71 primary vars. 2006 clauses. 29ms.
  No counterexample found. Assertion may be valid. 10ms.
```

Hình 4.5: Kết quả kiểm chứng assertion AcademicIntegrity

Điều này có nghĩa là trong phạm vi kiểm tra với 3 Student và tối đa 5 Course, Alloy không tìm thấy trạng thái nào vi phạm thuộc tính an toàn. Do đó có thể kết luận rằng mô hình thỏa mãn bất biến đã xác định trong không gian kiểm tra đã chọn.

4.1.4. Phân tích độ phức tạp

Giả sử n = Course là số lượng môn học trong hệ thống và m = Student là số lượng sinh viên. Thuật toán RegisterCourse kiểm tra các điều kiện tiên quyết trước khi cập nhật quan hệ enrolled của University.

Tiêu chí	Ký hiệu	Nội dung phân tích
Thời gian - Trường hợp tốt nhất	$\Omega(1)$	Môn học c không có môn tiên quyết (no c.prereq). Khi đó thuật toán kiểm tra điều kiện này ngay lập tức và cho phép đăng ký mà không cần duyệt thêm bất kỳ bước nào. Vòng lặp dừng ngay ở lần đầu tiên.
Thời gian - Trường hợp trung bình	$\Theta(d)$	d là độ sâu trung bình của chuỗi tiên quyết. Giả sử đồ thị tiên quyết phân bố ngẫu nhiên, thuật toán phải duyệt trung bình $d \approx \log(n)$ bước để xác định toàn bộ chuỗi prereq gián tiếp thông qua toán tử ^.
Thời gian - Trường hợp xấu nhất	$O(n)$	Trường hợp xấu nhất xảy ra khi chuỗi tiên quyết có dạng tuyến tính: $c_1 \rightarrow c_2 \rightarrow \ldots \rightarrow c_{\square}$. Khi đó toán tử bao đóng bắc cầu ^ phải duyệt toàn bộ chuỗi n môn học để xác định tất cả các môn tiên quyết gián tiếp của c.

Không gian bộ nhớ	O(m × n)	Thuật toán không tạo thêm cấu trúc dữ liệu mới. Bộ nhớ chủ yếu sử dụng để lưu trữ hai quan hệ nhị phân University.passed ⊆ Student × Course và University.enrolled ⊆ Student × Course, mỗi quan hệ có kích thước tối đa m × n.
Đánh giá mức độ tối ưu	Có thể tối ưu xuống Θ(1)	Để xác định sinh viên s đã pass môn tiên quyết hay chưa, hệ thống phải duyệt qua quan hệ passed và tính bao đóng bắc cầu của prereq. Nếu hệ thống bổ sung một chỉ mục ánh xạ trực tiếp từ (Student, Course) → trạng thái passed, việc kiểm tra có thể thực hiện bằng một phép tra cứu trực tiếp trong thời gian hằng số.

4.1.5. Mô hình hóa thuật toán trong Alloy

Trong Alloy, thay vì mô tả thuật toán bằng các vòng lặp như for hay while, hành vi của hệ thống được đặc tả ngắn gọn thông qua logic vị từ kết hợp với các ràng buộc logic, nhằm đảm bảo SAT Solver có thể mô phỏng chính xác các trạng thái hợp lệ của hệ thống.

```
// 1.định nghĩa
sig Student {}

sig Course {
prereq: lone Course // Một môn có tối đa (lone) một môn tiên quyết
}
// 2.Trạng thái hệ thống
one sig University {
passed: Student -> Course, // Sinh viên đã đậu môn nào
enrolled: Student -> Course // Sinh viên đang đăng ký môn nào
}
//3.ràng buộc logic
fact NoSelfPrereq {
// Một môn không thể là môn tiên quyết của chính nó (tránh vòng lặp)
no c: Course | c in c.^prereq // ^ là toán học bao đóng bắc cầu (transitive closure)
}

fact ValidRegistration {
// Quy tắc: Muốn đăng ký môn c, phải pass môn prereq của c
all s: Student, c: Course |
c in s.(University.enrolled) => (some p: c.prereq | p in s.(University.passed) or no c.prereq)
}
//4.ktra giả thuyết
assert AcademicIntegrity {
// Giả thuyết: Không sinh viên nào đang học một môn mà chưa pass môn tiên quyết
all s: Student, c: Course |
(c in s.(University.enrolled) and some c.prereq) => c.prereq in s.(University.passed)
}

// Lệnh thực thi
check AcademicIntegrity for 3 but 5 Course
```

Hình 4.4: Mô hình hóa hệ thống đăng ký học phần trong Alloy

Giải thích mã nguồn:

- Định nghĩa thực thể:
  - sig Student {}: khai báo một tập hợp các sinh viên. Mỗi Student là một nguyên tử (atom) trong Alloy.
  - sig Course { prereq: lone Course }: khai báo tập hợp các môn học, prereq là quan hệ giữa môn và môn tiên quyết, lone là mỗi môn có tối đa một môn tiên quyết.
- Trạng thái hệ thống:
  - one sig University nghĩa là trong mô hình chỉ có một thực thể đại diện cho trường học.
  - passed: Student -> Course là quan hệ cho biết sinh viên đã pass môn nào.
  - enrolled: Student -> Course là quan hệ cho biết sinh viên đang đăng ký môn nào.
- Ràng buộc logic:
  - no c: Course | c in c.^prereq nghĩa là không có môn nào là môn tiên quyết của chính nó, tránh vòng lặp (A là prereq của A, hoặc A → B → A).
  - all s: Student, c: Course | c in s.(University.enrolled) => (some p: c.prereq | p in s.(University.passed) or no c.prereq) nghĩa là muốn đăng ký môn c, phải pass môn prereq của c

4.2. Đặc tả ràng buộc

Để Alloy Analyzer có thể quét toàn bộ không gian trạng thái và tự động kiểm chứng các thuộc tính của hệ thống, bài toán đăng ký học phần được mô hình hóa dưới góc nhìn của Bài toán thỏa mãn ràng buộc (Constraint Satisfaction Problem – CSP).

4.2.1 Định nghĩa bài toán ràng buộc (CSP)

a, Tập biến: Thay vì xem hệ thống như một chuỗi các thao tác tuần tự, trạng thái sở hữu của mỗi cuốn sách được xem như một biến độc lập. Ta định nghĩa tập biến như sau:

$$X = \{ \text{enrollment}(s, c) \mid s \in \text{Student}, c \in \text{Course} \}$$

Trong đó:

enrollment(s, c) biểu diễn trạng thái đăng ký của sinh viên s đối với môn học c

b, Miền giá trị: Miền giá trị của mỗi biến được định nghĩa như sau:

$$D(\text{enrollment}(s, c)) = \{ \text{true}, \text{false} \}$$

Ý nghĩa:

- Nếu enrollment(s, c) = true → sinh viên s đang đăng ký môn c, tức là (s, c) ∈ University.enrolled
- Nếu enrollment(s, c) = false → sinh viên s không đăng ký môn c

c, Tập ràng buộc:

Các quy tắc nghiệp vụ của hệ thống được biểu diễn dưới dạng tập ràng buộc:

$$C = \{C1, C2\}$$

Trong đó:

- Ràng buộc C1 – NoSelfPrereq (Không có vòng lặp tiên quyết)
  Không tồn tại môn học nào là tiên quyết gián tiếp của chính nó:
  $$\nexists\, c \in Course : c \in c.\hat{}prereq$$
  Ràng buộc này đảm bảo hệ thống không có môn nào là tiên quyết của chính nó (A là prereq của A, hoặc A → B → A)

- Ràng buộc C2 – ValidRegistration (Điều kiện đăng ký hợp lệ)
  Sinh viên chỉ được đăng ký môn c khi đã vượt qua môn tiên quyết của c (nếu có):

  $\forall\, s \in Student, \forall\, c \in Course: enrollment(s, c) = true \rightarrow (pre(c) = \varnothing \vee (s, pre(c)) \in University.passed)$

  Ràng buộc này là cốt lõi của hệ thống, đảm bảo tính toàn vẹn học thuật (Academic Integrity) trong mọi trạng thái hợp lệ.

d, Hàm mục tiêu:

Hệ thống yêu cầu tìm các trạng thái hợp lệ hoặc phát hiện phản ví dụ, do đó bài toán được xem là bài toán thỏa mãn ràng buộc (CSP) thay vì bài toán tối ưu (COP). SAT Solver của Alloy sẽ tìm một phép gán giá trị:

$$A = \{ enrollment(s_1, c_1) \leftarrow v_1, enrollment(s_1, c_2) \leftarrow v_2, \ldots \}$$

Sao cho: $\forall\, c_i \in C : c_i(A) = True$, nghĩa là với mọi ràng buộc trong tập C, khi áp dụng lên phép gán A thì tất cả các ràng buộc đều được thỏa mãn.
Nếu không tồn tại phép gán nào vi phạm các ràng buộc, Alloy Analyzer sẽ báo: **No counterexample found**.

4.2.2 Phân loại ràng buộc

Loại ràng buộc	Mô tả	Ví dụ trong hệ thống	Alloy tương ứng
Unary	Giới hạn miền giá trị của một biến	Mỗi môn có tối đa một môn tiên quyết trực tiếp	prereq: lone Course
Transitive	Quan hệ bắc cầu qua nhiều cấp	Không có vòng lặp trong chuỗi A→B→C→A	no c: Course | c in c.^prereq
Implication	Ràng buộc điều kiện giữa hai quan hệ	Đăng ký môn c ⇒ đã pass prereq(c)	enrolled => prereq in passed

4.2.3 Lan truyền ràng buộc và tính nhất quán (Propagation & Consistency)

Khi Alloy Analyzer thực thi kiểm chứng, toàn bộ ràng buộc được chuyển đổi thành mệnh đề Boolean và giải bởi SAT Solver, áp dụng cơ chế Arc Consistency (AC) để giảm không gian tìm kiếm.

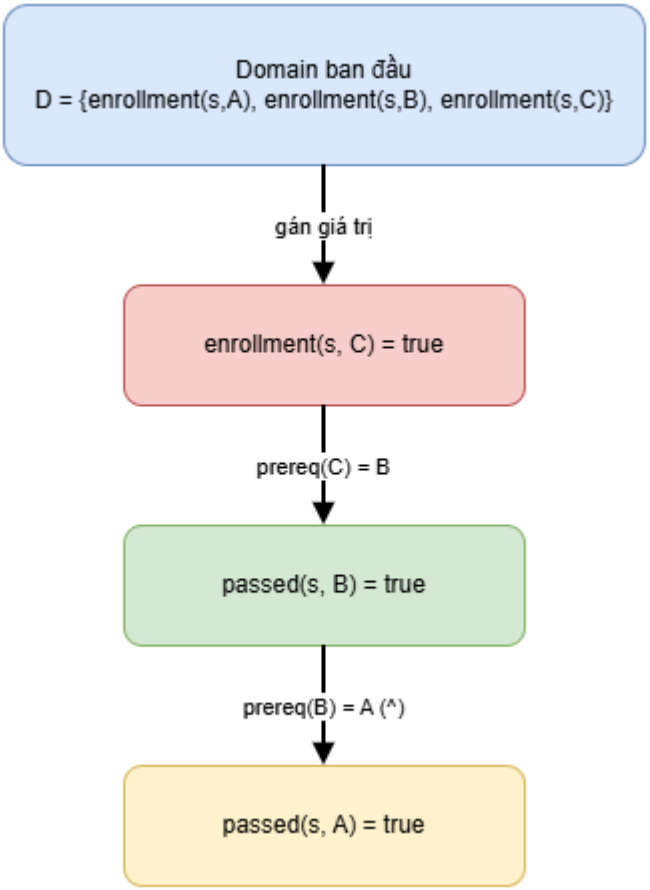
Ví dụ: xét sinh viên s muốn đăng ký môn C với chuỗi tiên quyết A→B→C. Giả sử miền giá trị ban đầu của trạng thái đăng ký là:

D = { enrollment(s, A), enrollment(s, B), enrollment(s, C) }

Nếu SAT Solver gán **enrollment(s, C) = true**, cơ chế Arc Consistency sẽ tự động lan truyền ràng buộc C2:

- enrollment(s, C) = true → passed(s, B) = true (B là prereq trực tiếp của C)
- passed(s, B) = true → passed(s, A) = true (A là prereq trực tiếp của B, phát hiện qua toán tử ^)

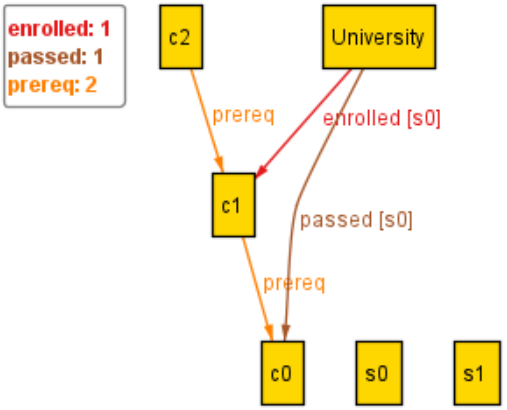
➔ Nhờ cơ chế lan truyền này, không gian tìm kiếm của SAT Solver được cắt giảm đáng kể theo từng bước, giúp Alloy Analyzer nhanh chóng phát hiện các trạng thái vi phạm hoặc chứng minh rằng mô hình không chứa phản ví dụ trong phạm vi kiểm chứng đã chọn.



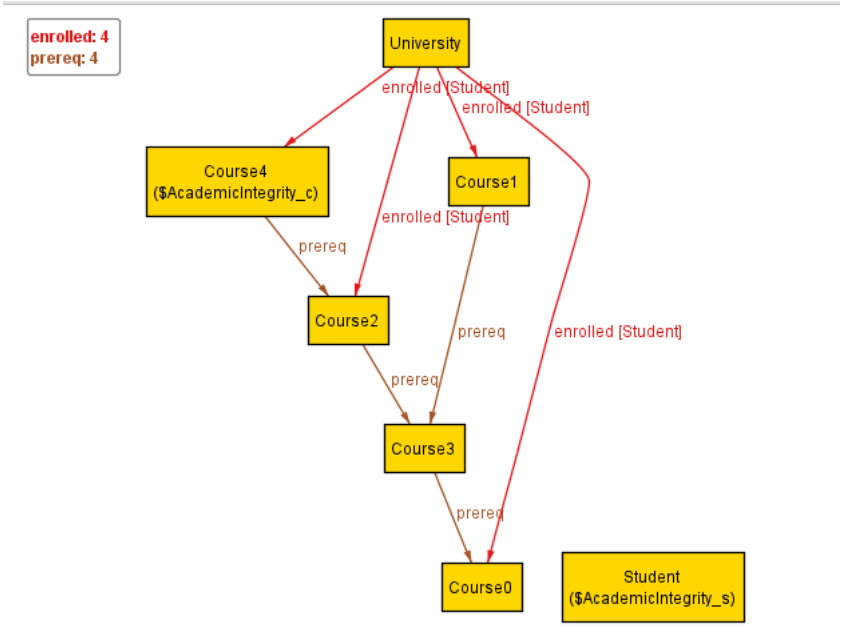
Hình 4.5: Đồ thị ràng buộc

Chương 5: TÌNH HUỐNG (SCENARIOS) VÀ KIỂM CHỨNG

5.1. Thiết kế tình huống kiểm chứng

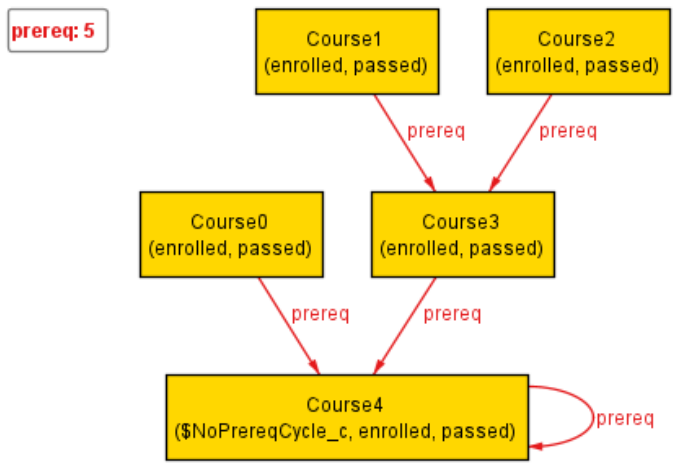
ID & Tên	S-N-01: Đăng ký học phần hợp lệ
Thuộc nhóm	Nominal (Happy Path)
Mục đích	Kiểm tra predicate registerCourse hoạt động đúng khi sinh viên đăng ký môn có prereq đã pass.
Thiết lập (Setup)	• 2 Student: s0, s1 • 3 Course: c0 (không prereq), c1 (prereq: c0), c2 (prereq: c1) • s0 đã pass c0 | s0 muốn đăng ký c1
Lệnh Alloy	run {} for 2 Student, 3 Course
Kết quả thực tế	• Student = {s0, s1} • Course = {c0, c1, c2} • s0.passed = {c0} • s0.enrolled = {c1}
Phân tích	instance cho thấy: • s0 có c0 trong passed và c1 trong enrolled. • s1 chỉ có thể đăng ký c0 vì chưa pass prereq.
Ảnh minh họa	

Bảng 5.1: Tình huống S-N-01 – Đăng ký hợp lệ

ID & Tên	S-A-01: vi phạm AcademicIntegrity (Fault Injection)
Thuộc nhóm	Adversarial – Fault Injection
Mục đích	Kỹ thuật fault injection: cố ý xóa fact ValidRegistration để Alloy tự sinh counterexample, minh họa điều gì xảy ra nếu thiếu ràng buộc hình thức
Thiết lập (Setup)	• Xóa fact ValidRegistration khỏi mô hình. • Giữ nguyên NoSelfPrereq.
Lệnh Alloy	check AcademicIntegrity for 3 but 5 Course
Kết quả thực tế	Xuất hiện Counterexample - Đúng với kết quả kỳ vọng
Phân tích	Counterexample trực quan chứng minh: nếu thiếu ràng buộc ValidRegistration, hệ thống sẽ cho phép đăng ký bất kỳ môn nào mà không kiểm tra tiên quyết. Đây là bằng chứng mạnh mẽ về giá trị của phương pháp hình thức.
Ảnh minh họa	

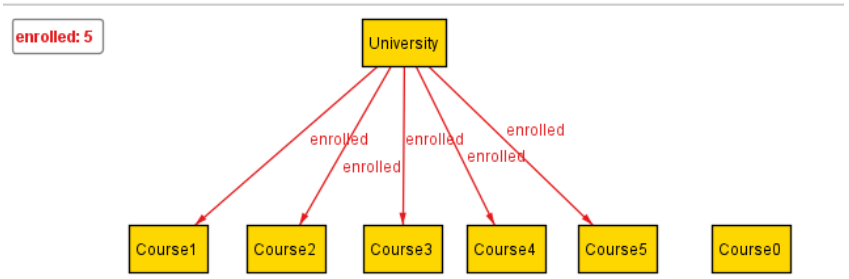
Bảng 5.2: Tình huống S-A-01 – Fault Injection, vi phạm AcademicIntegrity

ID & Tên	S-A-02: Vòng lặp tiên quyết
Thuộc nhóm	Adversarial

Mục đích	Kiểm tra rằng fact NoSelfPrereq ngăn chặn vòng lặp A→B→C→A trong đồ thị tiên quyết.
Thiết lập (Setup)	• Xóa fact NoSelfPrereq khỏi mô hình. • Giữ nguyên ValidRegistration.
Lệnh Alloy	check NoPrereqCycle for 3 Student, 5 Course
Kết quả thực tế	Alloy sinh counterexample với chu trình: Course0.prereq = Course2 Course2.prereq = Course1 Course1.prereq = Course1 (tự phụ thuộc) → Tạo thành vòng lặp tiên quyết giữa các môn học.
Phân tích	Alloy phát hiện được chu trình trong quan hệ prereq. Điều này cho thấy nếu không có ràng buộc NoSelfPrereq thì mô hình cho phép các môn học tạo vòng lặp tiên quyết, khiến sinh viên không thể hoàn thành chuỗi môn học. Đây là ví dụ cho thấy Alloy có thể tự động tìm lỗi logic trong mô hình.
Ảnh minh họa	

Bảng 5.3: Tình huống S-A-02 – Vòng lặp tiên quyết

ID & Tên	Tình huống S-B-01: Giới hạn số môn đăng ký
Thuộc nhóm	Boundary
Mục đích	Kiểm chứng fact CreditLimit ngăn sinh viên đăng ký quá 5 môn.
Thiết lập (Setup)	• 2 Student: s0, s1

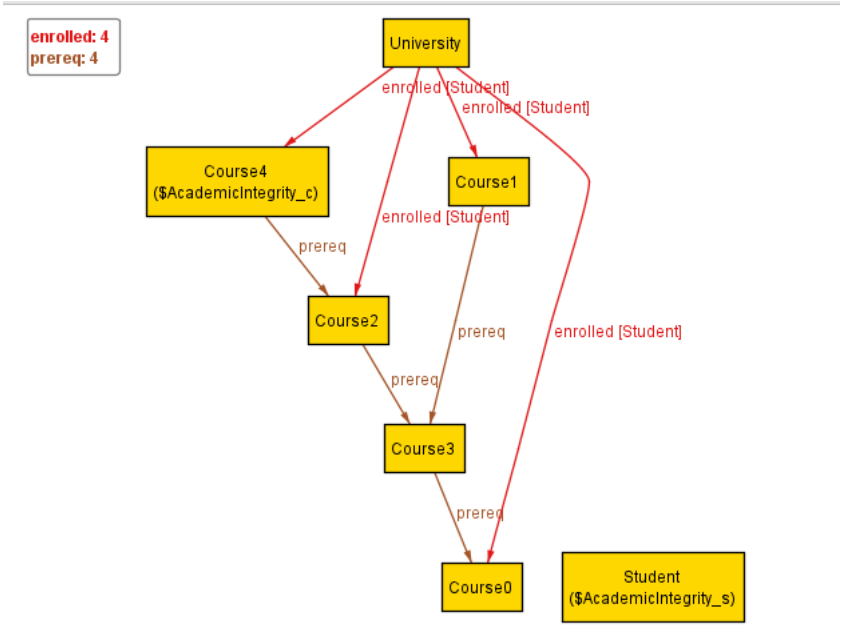
	• 6 Course: c0, c1, c2, c3, c4, c5 • s0 đăng ký các môn học trong hệ thống
Lệnh Alloy	check AcademicIntegrity for 5 Student, 10 Course
Kết quả thực tế	• Student = {s0, s1} • Course = {c0, c1, c2, c3, c4, c5} • s0.enrolled = {c0, c1, c2, c3, c4}
Phân tích	Instance cho thấy sinh viên s0 chỉ đăng ký tối đa 5 môn học. Alloy không tạo trường hợp sinh viên đăng ký nhiều hơn 5 môn do ràng buộc LimitCourseRegistration đã giới hạn số môn đăng ký.
Ảnh minh họa	

Bảng 5.3: : Tình huống S-B-01 – Giới hạn số môn đăng ký

5.2. Phân tích Counterexample

<u>Counterexample 1: Vi phạm AcademicIntegrity (Fault Injection)</u>

Khi xóa fact ValidRegistration, Alloy sinh counterexample và tìm được instance sau:



Hình 5.1: Counterexample – Student đăng ký môn chưa pass prereq (Alloy Visualizer)

Trong instance này:

University.enrolled = {(s, Course4), (s, Course2), (s, Course1), (s, Course0)} và University.passed = {} (rỗng).

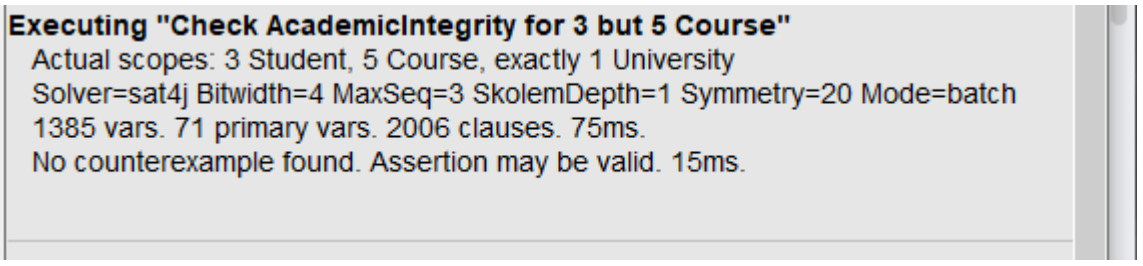
Quan hệ tiên quyết giữa các môn là:

- Course4.prereq = Course2
- Course2.prereq = Course3
- Course3.prereq = Course0
- Course1.prereq = Course3

Sinh viên s đang đăng ký nhiều môn học mặc dù chưa pass bất kỳ môn tiên quyết nào trong chuỗi này. Ví dụ, sinh viên đăng ký Course0 trong khi vẫn chưa hoàn thành Course3, là môn tiên quyết của nó.

Nguyên nhân: Thiếu ràng buộc fact ValidRegistration, nên Alloy không bị giới hạn kiểm tra rằng sinh viên phải đã pass tất cả các môn tiên quyết trước khi đăng ký môn mới, dẫn đến việc các cặp (s, Course) vẫn có thể xuất hiện trong enrolled.

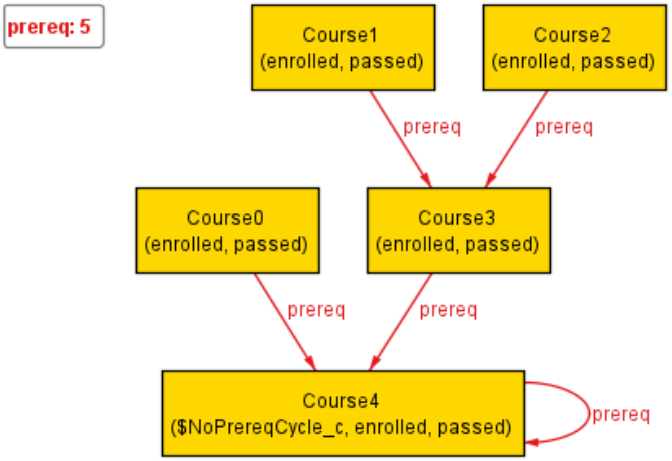
➔ Khắc phục: Thêm lại fact ValidRegistration. Sau đó kết quả sẽ trả về:



Hình 5.2: Kết quả kiểm chứng sau khi khôi phục ValidRegistration

<u>Counterexample 2: Vòng lặp tiên quyết</u>

Khi xóa fact NoSelfPrereq, Alloy sinh counterexample và tìm được instance sau:



Hình 5.3: Counterexample – Vòng lặp tiên quyết

Trong instance này:

- Course1.prereq = Course3
- Course2.prereq = Course3
- Course3.prereq = Course4
- Course0.prereq = Course4
- và Course4.prereq = Course4.

Toán tử bao đóng bắc cầu ^ phát hiện rằng **Course4 ∈ Course4.^prereq**, nghĩa là môn học phụ thuộc vào chính nó thông qua quan hệ tiên quyết. Điều này **vi phạm ràng buộc NoPrereqCycle**. Nếu chỉ kiểm tra quan hệ prereq trực tiếp mà không dùng toán tử ^, thì các chuỗi phụ thuộc dài hoặc vòng lặp như vậy có thể **không được phát hiện**.

➔ Khắc phục: Thêm lại fact NoSelfPrereq. Sau đó kết quả sẽ trả về:

```
Executing "Check NoPrereqCycle for 3 Student, 5 Course, exactly 1 University"
  Actual scopes: 3 Student, 5 Course, exactly 1 University
  Solver=sat4j Bitwidth=4 MaxSeq=4 SkolemDepth=1 Symmetry=20 Mode=batch
  1256 vars. 68 primary vars. 2614 clauses. 186ms.
  No counterexample found. Assertion may be valid. 23ms.
```

Hình 5.4: Kết quả kiểm chứng sau khi khôi phục NoSelfPrereq.

5.3. Phân tích Scope và giới hạn kiểm chứng

Trong Alloy, việc kiểm chứng mô hình được thực hiện trong một phạm vi hữu hạn (scope). Scope xác định số lượng tối đa các đối tượng mà Alloy sẽ tạo cho mỗi signature. Việc lựa chọn scope ảnh hưởng trực tiếp đến độ bao phủ của kiểm chứng và thời gian chạy của bộ giải SAT.

Trong quá trình kiểm chứng mô hình đăng ký học phần, các lệnh check được thực hiện với nhiều giá trị scope khác nhau để đánh giá độ ổn định của mô hình.

Ví dụ: check AcademicIntegrity for 3 Student, 5 Course

check AcademicIntegrity for 5 Student, 7 Course

check AcademicIntegrity for 10 Student, 12 Course

Trong đó:

- for 3 Student, 5 Course nghĩa là Alloy tạo tối đa 3 đối tượng Student và tối đa 5 đối tượng Course trong quá trình tìm kiếm instance.
- Scope nhỏ giúp Alloy tìm counterexample nhanh hơn vì số trạng thái cần kiểm tra ít hơn.
- Scope lớn giúp kiểm chứng mô hình trong nhiều trường hợp hơn, từ đó tăng độ tin cậy rằng mô hình không có lỗi logic.

Kết quả kiểm chứng theo scope

Scope	Thời gian chạy	instance tìm được	kết quả
3 Student, 5 Course	~17ms	Có	Không tìm thấy counterexample
5 Student, 7 Course	~21ms	Có	Không tìm thấy counterexample
10 Student, 12 Course	~47ms	Có	Không tìm thấy counterexample

Bảng 5.4: Kết quả kiểm chứng mô hình theo các giá trị Scope khác nhau trong Alloy

Khi tăng scope, Alloy phải kiểm tra số lượng cấu hình lớn hơn rất nhiều. Điều này dẫn đến độ bao phủ kiểm chứng cao hơn, nhưng thời gian chạy tăng nhanh theo cấp số mũ. Đây là đặc trưng của các công cụ SAT-based model checking như Alloy.

<u>Nguyên lý Small Scope Hypothesis</u>

Alloy dựa trên nguyên lý Small Scope Hypothesis, cho rằng: "Nếu một mô hình có lỗi, lỗi đó thường xuất hiện trong các instance nhỏ." Việc kiểm chứng trong scope nhỏ (3--5 đối tượng) vẫn có khả năng phát hiện phần lớn các lỗi logic quan trọng trong mô hình.

Nguyên lý này đặc biệt phù hợp với bài toán đăng ký học phần vì các lỗi điển hình như vòng lặp tiên quyết hay vi phạm điều kiện prereq đều có thể được phát hiện trong scope rất nhỏ. Cụ thể:

- Vòng lặp tiên quyết 2 cấp (A→B→A) đã đủ để thể hiện lỗi, không cần chuỗi 100 cấp
- Vi phạm AcademicIntegrity chỉ cần 1 Student và 2 Course (một môn có prereq, sinh viên chưa pass) là đủ để Alloy sinh counterexample.

- Vi phạm CreditLimit chỉ cần 1 Student và 7 Course không có prereq là đủ để phát hiện.

Trong mô hình đăng ký học phần, các lỗi như:

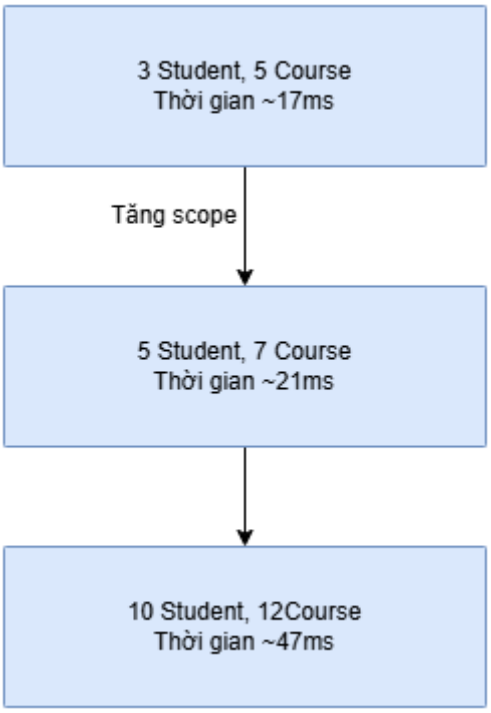
- Sinh viên đăng ký môn chưa pass prereq.
- Vòng lặp trong đồ thị tiên quyết (A→B→C→A).
- Sinh viên đăng ký quá 5 môn trong một học kỳ.

đều có thể được phát hiện trong scope nhỏ (3--5 đối tượng), phù hợp với nguyên lý trên.

<u>Giới hạn của kiểm chứng theo scope</u>

Mặc dù Alloy rất hiệu quả trong việc tìm counterexample, cần lưu ý rằng kiểm chứng chỉ được thực hiện trong phạm vi scope hữu hạn. Alloy không cung cấp chứng minh toán học cho mọi trường hợp vô hạn. Do đó, kết quả "No counterexample found" chỉ có nghĩa là không tìm thấy lỗi trong phạm vi đã kiểm tra, chứ không đảm bảo mô hình đúng tuyệt đối trong mọi trường hợp có thể xảy ra.

Đây cũng là lý do đồ án kết hợp cả hai phương pháp: kiểm chứng tự động bằng Alloy Analyzer để phát hiện lỗi nhanh chóng trong phạm vi hữu hạn, và chứng minh toán học bằng bất biến (trình bày ở mục 4.1.3) để đảm bảo tính đúng đắn về mặt lý thuyết.



Hình 5.5:Ảnh hưởng của Scope đến thời gian kiểm chứng trong Alloy

Chương 6: Kết quả, thảo luận và hạn chế

6.1 Tổng hợp kết quả kiểm

ID	Assertion	Thuộc kiểm chứng	Scope	Kết quả	Ý nghĩa
A-01	RegistrationSafety	Một học phần không thể được đăng ký nhiều lần bởi cùng một sinh viên	5	UNSAT	Đảm bảo sinh viên không thể đăng ký trùng một học phần
A-02	CourseLoadLimitCheck	Sinh viên không thể đăng ký quá 5 học phần	5	UNSAT	Đảm bảo tuân thủ quy định về giới hạn số học phần tối đa
A-03	NoDuplicateRegistrationCheck	Một sinh viên không thể đăng ký cùng một học phần nhiều lần	5	UNSAT	Ngăn chặn việc đăng ký trùng lặp học phần
A-04	RegisterOnlyIfValidCourseCheck	Sinh viên chỉ có thể đăng ký các học phần tồn tại trong hệ thống	5	UNSAT	Đảm bảo tính hợp lệ của việc đăng ký học phần
A-05	CoreCourseRegistrationCheck	Các học phần bắt buộc phải được đăng ký bởi ít nhất một sinh viên	5	UNSAT	Đảm bảo các học phần bắt buộc được tham gia học

Bảng 6.1 Bảng tổng hợp kết quả kiểm chứng

6.2 Thảo luận sau kết quả

Kết quả kiểm chứng bằng Alloy cho thấy mô hình hệ thống đăng ký học phần đã đáp ứng các ràng buộc nghiệp vụ quan trọng. Các assertion được kiểm tra đều không tìm thấy counterexample trong phạm vi scope đã chọn, cho thấy mô hình đảm bảo tính nhất quán và an toàn của dữ liệu trong các trường hợp được kiểm tra.

Trong quá trình kiểm chứng ban đầu, Alloy đã phát hiện một số counterexample khi các ràng buộc chưa được mô tả đầy đủ. Ví dụ, có thể xuất hiện trường hợp một sinh viên đăng ký quá số lượng học phần cho phép hoặc đăng ký trùng một học phần nhiều lần. Việc phát hiện các counterexample này giúp xác định những thiếu sót trong mô hình và bổ sung các ràng buộc cần thiết như giới hạn số học phần tối đa và ngăn chặn đăng ký trùng lặp.

Sau khi cập nhật các ràng buộc phù hợp, Alloy không còn tìm thấy các phản ví dụ trong phạm vi kiểm tra, cho thấy mô hình đã đáp ứng được các quy tắc nghiệp vụ cơ bản của hệ thống.

Tuy nhiên, mô hình hiện tại vẫn còn một số hạn chế. Chẳng hạn, mô hình chưa xét đến các yếu tố như thời gian đăng ký học phần, số lượng sinh viên tối đa của mỗi lớp hoặc điều kiện tiên quyết của môn học. Ngoài ra, việc kiểm chứng của Alloy chỉ được thực hiện trong phạm vi scope hữu hạn nên kết quả không phải là chứng minh tuyệt đối cho mọi trường hợp.

Mặc dù vậy, việc sử dụng Alloy trong đề tài đã cho thấy hiệu quả của phương pháp mô hình hóa hình thức trong việc phát hiện sớm các lỗi logic và hỗ trợ kiểm tra tính đúng đắn của hệ thống trước khi triển khai thực tế.

Chương 7. Kết luận và hướng phát triển

7.1 Kết luận nghiên cứu

Đồ án đã áp dụng phương pháp hình thức (Formal Methods) để đặc tả và kiểm chứng chức năng đăng ký học phần trong hệ thống quản lý đào tạo thông qua ngôn ngữ Alloy và công cụ Alloy Analyzer. Thông qua việc xây dựng mô hình, định nghĩa các ràng buộc của hệ thống và tiến hành kiểm chứng tự động, nghiên cứu đã đánh giá khả năng phát hiện lỗi logic cũng như hiệu quả của phương pháp kiểm chứng dựa trên mô hình. Dựa trên các kết quả thực nghiệm, các câu hỏi nghiên cứu được đặt ra ở phần đầu của đề tài được trả lời như sau:

Câu hỏi nghiên cứu	Kết luận từ thực nghiệm
RQ1: Alloy có thể được sử dụng để mô hình hóa các thực thể và mối quan hệ trong hệ thống đăng ký học phần hay không?	Có. Alloy cho phép mô hình hóa rõ ràng các thực thể như Student, Course và mối quan hệ đăng ký giữa chúng thông qua các signature và relation. Các ràng buộc logic được biểu diễn bằng fact, predicate và assertion giúp mô hình phản ánh chính xác quy tắc hoạt động của hệ thống.
RQ2: Alloy Analyzer có thể phát hiện các vi phạm ràng buộc trong hệ thống hay không?	Có. Alloy Analyzer có thể tự động tìm kiếm counterexample khi một ràng buộc bị vi phạm. Khi các ràng buộc bị loại bỏ hoặc thay đổi, công cụ đã phát hiện các trường hợp như sinh viên đăng ký quá số học phần cho phép hoặc đăng ký trùng học phần.
RQ3: Việc thay đổi scope ảnh hưởng như thế nào đến quá trình kiểm chứng?	Khi scope tăng, số lượng trạng thái cần kiểm tra tăng lên đáng kể, làm thời gian kiểm chứng tăng. Tuy nhiên trong phạm vi nhỏ (ví dụ ≤5), Alloy vẫn phát hiện hiệu quả các lỗi logic quan trọng của mô hình, phù hợp với giả thuyết Small Scope Hypothesis.

Bảng 7.1 Câu hỏi nghiên cứu và kết luận từ thực tế

Nhìn chung, kết quả nghiên cứu cho thấy Alloy là công cụ hiệu quả trong việc mô hình hóa và kiểm chứng các ràng buộc logic của hệ thống trong giai đoạn phân tích và thiết kế phần mềm.

7.2 Đóng góp của đề tài

7.2.1 Đóng góp đề tài

Đề tài đã minh họa cách áp dụng phương pháp hình thức để đặc tả và kiểm chứng hệ thống đăng ký học phần. So với các công cụ hình thức phức tạp hơn, Alloy có ưu điểm là dễ tiếp cận, trực quan và phù hợp với phương pháp hình thức hạng nhẹ.

Về mặt thực tiễn, đồ án đã xây dựng mô hình Alloy cho hệ thống đăng ký học phần bao gồm:

- Đặc tả các thực thể chính của hệ thống (Student, Course)
- Quan hệ đăng ký học phần (registers)
- Các ràng buộc toàn vẹn của hệ thống
- Các assertion kiểm chứng và kịch bản kiểm tra

Mô hình này có thể được sử dụng làm tài liệu tham khảo cho việc học tập phương pháp hình thức cũng như hỗ trợ quá trình phân tích và thiết kế hệ thống phần mềm.

7.2.2 Hạn chế của nghiên cứu

- Mô hình tĩnh: chưa mô tả đầy đủ state transitions như hủy đăng ký, điều chỉnh lịch học.
- Prereq đơn giản: mỗi môn chỉ có tối đa một prereq trực tiếp (lone), trong thực tế có thể có nhiều prereq.
- Chưa mô hình hóa điểm số: "đã pass" được giả định là thuộc tính boolean, chưa xét ngưỡng điểm cụ thể.
- Scope giới hạn: kết quả "No counterexample found" không phải chứng minh toán học tuyệt đối.

7.3 Hướng phát triển trong tương lai

Trong các nghiên cứu tiếp theo, mô hình có thể được mở rộng theo các hướng sau:

- Mô hình hóa theo thời gian: Mô hình có thể được mở rộng để mô phỏng sự thay đổi trạng thái của hệ thống theo thời gian, chẳng hạn như:
    + Thêm học phần
    + Hủy đăng ký học phần
    + Điều chỉnh đăng ký

  Điều này giúp mô hình phản ánh đầy đủ hơn hoạt động thực tiễn của hệ thống.

- Mở rộng mô hình quản lý học phần: Một hướng phát triển khác là bổ sung các ràng buộc như:
    + Điều kiện tiên quyết của học phần
    + Phân loại các học phần bắt buộc và tự chọn

- Tích hợp vào quy trình phát triển phần mềm: Trong tương lai, các ràng buộc hình thức trong Alloy có thể được chuyển thành các cơ chế kiểm tra trong hệ thống thực tế như:
    + Unit Test trong các ngôn ngữ lập trình
    + Ràng buộc toàn vẹn trong cơ sở dữ liệu

Việc tích hợp các kiểm chứng này vào quy trình phát triển phần mềm sẽ giúp phát hiện lỗi thiết kế sớm hơn và nâng cao chất lượng hệ thống.

Tài Liệu tham khảo

[1] FORMAL METHODS SPECIFICATION AND ANALYSIS GUIDEBOOK FOR THE VERIFICATION OF SOFTWARE AND COMPUTER SYSTEMS

[2] LECTURE 6: INTRODUCTION TO FORMAL METHODS - GIÁO TRÌNH GIẢNG DẠY MÔN HỌC SOFTWARE ENGINEERING CỦA GIÁO SƯ MIKE WOOLDRIDGE TẠI ĐẠI HỌC OXFORD

[3] Formal validation for microgrid reliability based on TLA+ (Muhammad Nasar, János Csatár)

[4] A Systematic Literature Review on a Decade of Industrial TLA+ Practice⋆

[5] A Certified Compiler for Verifiable Computing

[6] Aluminum: Principled Scenario Exploration through Minimality

[7] Designing and Analyzing a Flash File System with Alloy

[8] Experiences of Teaching a Lightweight Formal Method

[9] Formal Modeling and Verification of Smart Contracts with Spin

[10] seL4: formal verification of an OS kernel

[11] Alloy Analyzer